

VOLUME 11 / RUST BACKEND MASTERY

品質保証 — テスト・セキュリティ・可観測性・性能

何を、どの段のテストで確かめるかを決め、ユニットテストから負荷試験・CI までを手を動かして身に付ける

第11巻: 品質保証 — テスト・セキュリティ・可観測性・性能

この巻で身に付けること

第10巻までで、RustNote の画面と機能は、ひととおりそろいました。けれども、「今日、動いた」ことと、「明日も、誰かが直したあとも、ちゃんと動く」ことは別の話です。次のような事故は、動くアプリでも起きます。

- 小さな修正のつもりが、別の画面を壊していたのに、公開するまで誰も気づかなかった
- 英語のデータでは問題が無かったのに、日本語のタイトルを入れたら弾かれた
- 他人のノートが見えてしまう不具合を一度直したのに、半年後に同じ不具合がまた入った
- 公開したら遅くなったが、どこが遅いのか、測る方法が無かった
- 夜中に止まっていたのに、朝、利用者から言われるまで分からなかった

この巻では、こうした事故を、**テスト・見張り（可観測性）・測定**で、早く・自動で見つける方法を学びます。大事なのは、「とにかくテストをたくさん書く」ことではなく、**何を、どの段のテストで確かめるか**を決めることです。

到達目標

この巻を終えると、次のことができるようになります。

- 境界の値（空・1・最大・最大 + 1・日本語・空白）を表に並べた、ユニットテストを書ける。テストが本当に間違いを見つけられるかを、わざと壊した版で確かめられる
- 本物の DB を使った結合テスト、HTTP の返事（状態コード・ヘッダー・本文・クッキー）のテスト、ブラウザで操作する E2E テストを、役割を分けて書ける
- 脅威モデル（第8巻）を、「攻撃が止まること」を確かめる守りのテストに変えられる
- ログ・数の記録（メトリクス）・ヘルスチェックの違いを説明し、速さを release ビルドで測り、負荷をかけて限界を調べられる
- CI で、整形・Clippy・テスト・依存の監査を、毎回自動で回せる

学習地図

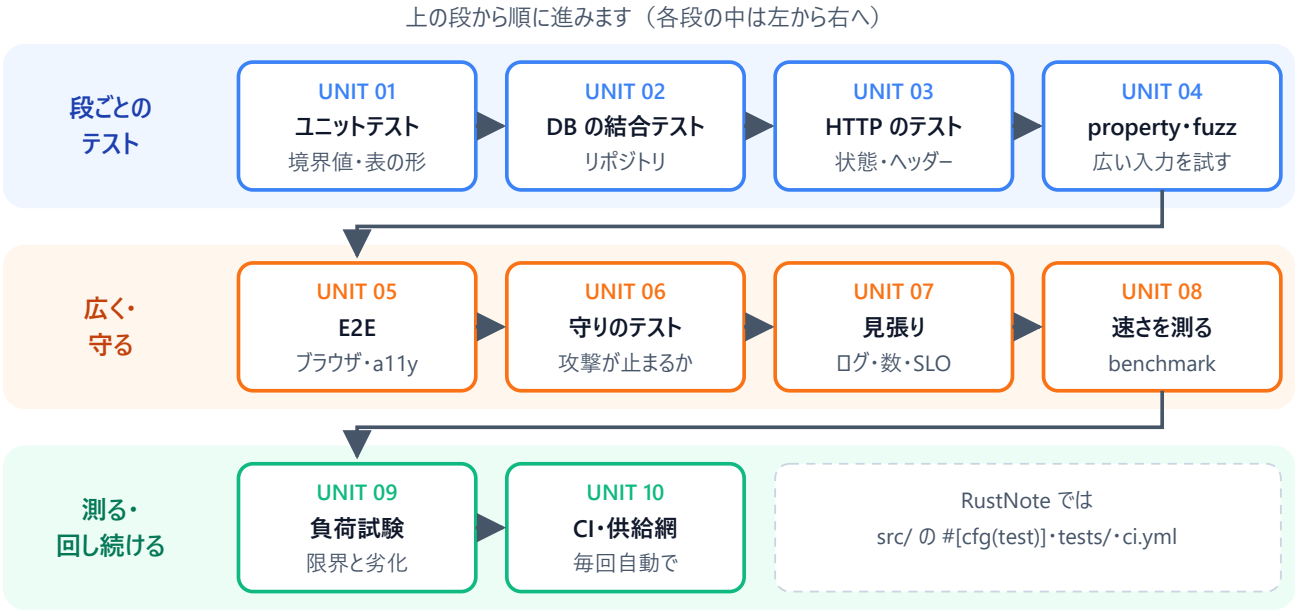


図11-1: 第11巻の学習地図。小さな段のテストから順に積み上げ、広い入力と攻撃を試し、速さと動きを測って、CI で回し続ける

単元	テーマ	ひとことで言うと
UNIT 01	ユニットテスト・境界値・表の形のテスト	端の値を表に並べ、関数1つを速く確かめる
UNIT 02	リポジトリの結合テスト	本物の DB と SQL で、保存と読み出しを確かめる
UNIT 03	HTTP のテスト	状態コード・Location・クッキー・本文を確かめる
UNIT 04	property test・fuzz・入力の読み取り	思いつかない入力を、たくさん自動で試す
UNIT 05	E2E・ブラウザ・アクセシビリティ（準備中）	大事な流れだけを、本物のブラウザで確かめる
UNIT 06	守りの回帰テスト（準備中）	攻撃が止まることを、毎回確かめる
UNIT 07	tracing・メトリクス・health・SLO（準備中）	動いている様子を外から見えるようにする
UNIT 08	benchmark・profile・メモリの使い方（準備中）	遅いところを、測ってから直す
UNIT 09	負荷試験・背圧・容量（準備中）	たくさん来たときの限界と、壊れ方を決める
UNIT 10	CI・依存・供給網・事故の対応（準備中）	毎回同じ確かめを自動で回し、部品の安全も見る

> **お知らせ:** この版には UNIT 01〜04 を収録しています。UNIT 05〜10 は準備中で、次の更新から順に追加します。

前提知識（先に読んでおく巻）

- **第2巻（Rust 基礎）**：日本語の文字数を `chars().count()` で数えること（第2巻 UNIT 05）。UNIT 01 の表に、日本語と絵文字の行が出てきます。
- **第4巻（設計）**：検査済みの型（第4巻 UNIT 01）と、テストの書き方の基本（`#[test]`・`assert!`・`assert_eq!`。第4巻 UNIT 10）。
- **第9巻（RustNote 設計）**：RustNote のドメインの型（`NoteTitle`・`Email`・`Password`・`NoteId`）と、その決まり（第9巻 UNIT 03）。この巻のテストも、第9巻と同じ形（`tests/` に足して `cargo test --test 名前`）で書きます。

この巻の進め方

テストは、「通った」だけでは、本当に役に立っているかが分かりません。各単元では、次の順番で進めてください。

1. **決まりを書き出す:** テストを書く前に、確かめる決まり（「タイトルは 1〜80 文字」など）を、日本語で書き出します。
2. **端の値を並べる:** 決まりの両側（80 と 81、0 と 1）と、間違えやすい入力（空白・日本語・絵文字）を表にします。
3. **テストを動かす:** 全部の行が通ることを確かめます。
4. **わざと壊して確かめる:** 関数をわざと間違えた版に変えても、テストがちゃんと失敗するかを確かめます。失敗しないテストは、何も守っていません。

使うツールと版

この巻のコードは、すべて次の環境で実際にコンパイル・実行し、実行結果とエラーメッセージは実際の出力をそのまま載せています（パソコンのユーザー名などを含む部分は `...` に置き換えています）。

項目	版・内容
OS	Windows 11（ARM64）。コマンドは PowerShell を主に、WSL / macOS / Linux は補足で示します
Rust	**1.99 以上**（本書シリーズの基準。Topcoat 0.11 が 1.99 以上を求めます）。動作確認は rustc 1.99.1 / cargo 1.99.1（`aarch64-pc-windows-msvc`）
RustNote	教材の `Rust/rustnote/`（`rustnote` 0.1.0）。この巻では `src/` を変えずに、`tests/` と `examples/` に確かめ用のファイルを足す（UNIT 03 からは、練習用のコピーの `Cargo.toml` の dev-dependencies にも足す）
scraper	0.29.0（UNIT 03。HTML を読んで、CSS セレクターで要素を探す）
proptest	1.12.0（UNIT 04。property test）
cargo-fuzz	0.13.2（UNIT 04。WSL の Ubuntu 24.04（aarch64）で、rustc 1.101.0-nightly とともに使う）
テストの道具	Rust の標準の `#[test]`・`assert!`・`assert_eq!`（UNIT 01 では、外のクレートを足しません）

練習用のフォルダを用意する

第9巻と同じように、教材の `Rust/rustnote/` を**作業用のフォルダにコピー**して使います。

```
# 教材の rustnote フォルダを、中身ごと（-Recurse）作業用のフォルダへコピーする
Copy-Item -Recurse "教材のフォルダ\Rust\rustnote" .\rustnote
cd rustnote
# このフォルダで使われる Rust の版を確かめる（1.99.1 になる）
rustup show active-toolchain
```

UNIT 01: ユニットテスト・境界値・表の形のテスト

この単元のゴール

- テストの「段」（ユニット・結合・HTTP・E2E）の違いを、速さと確かめる範囲で説明できる
- 決まりの両側の値（境界値: 0 と 1、80 と 81）と、間違えやすい入力（空白・全角の空白・日本語・絵文字）を表に並べて、1つのテストで確かめられる
- わざと間違えた版の関数に同じ表を当てて、テストが本当に間違いを見つけられるかを確かめられる
- 文字列の長さの3つの数え方（文字・バイト・UTF-16）の違いを、実際の値で説明できる

たとえ話でつかむ

お菓子の工場で、袋詰めの機械を検査する場面を考えます。決まりは「1袋に 1〜80 個」です。

- 検査の人は、40 個のような**ちょうど真ん中**の袋だけを試すことはしません。真ん中が正しくても、端で間違える機械はよくあるからです。**0 個・1 個・80 個・81 個**の4つを必ず試します。決まりの**境目の両側**です。
- 形が崩れたお菓子・小さなお菓子・2つがくっついたお菓子のように、**数え方を間違えやすいもの**も試します。
- 検査の結果は、**表**に書きます。「何を入れたか・どうなるはずか・どうなったか」を1行ずつ並べれば、どこで間違えたかが一目で分かります。
- そして、検査の表そのものが正しいかも確かめます。わざと**壊した機械**に同じ検査をして、ちゃんと「不合格」が出るかを見るのです。壊れた機械まで合格にしてしまう検査は、意味がありません。

お菓子の工場	RustNote のテスト
「1袋に 1〜80 個」の決まり	`NoteTitle::parse` の「前後の空白を除いて 1〜80 文字」
0・1・80・81 個の袋	**境界値**（決まりの端と、その外側）
数え方を間違えやすいお菓子	全角の空白・日本語・絵文字・濁点を分けて書いた「が」
検査の表	**表の形のテスト**（入力と期待する結果を、1行ずつ並べたテスト）
壊した機械で検査を試す	**わざと間違えた版**で、テストが失敗するかを見る

> **用語:** ユニットテスト（unit test：関数や型のような小さな部品1つを、ほかから切り離して確かめるテスト。DB もネットワークも使わないので速い）

> **用語:** 境界値（きょうかいち：決まりが切り替わる境目の値。「1〜80 文字」なら、0・1・80・81 文字）

> **用語:** 表の形のテスト（ひょうのかたちのテスト。table-driven test：入力と期待する結果の組を表（配列）に並べ、同じ確かめ方で1行ずつ試すテスト）

図で理解する

テストには、確かめる範囲の違う「段」があります。RustNote にもともと入っている 41 件のテストを、段ごとに分けると次のようになります。

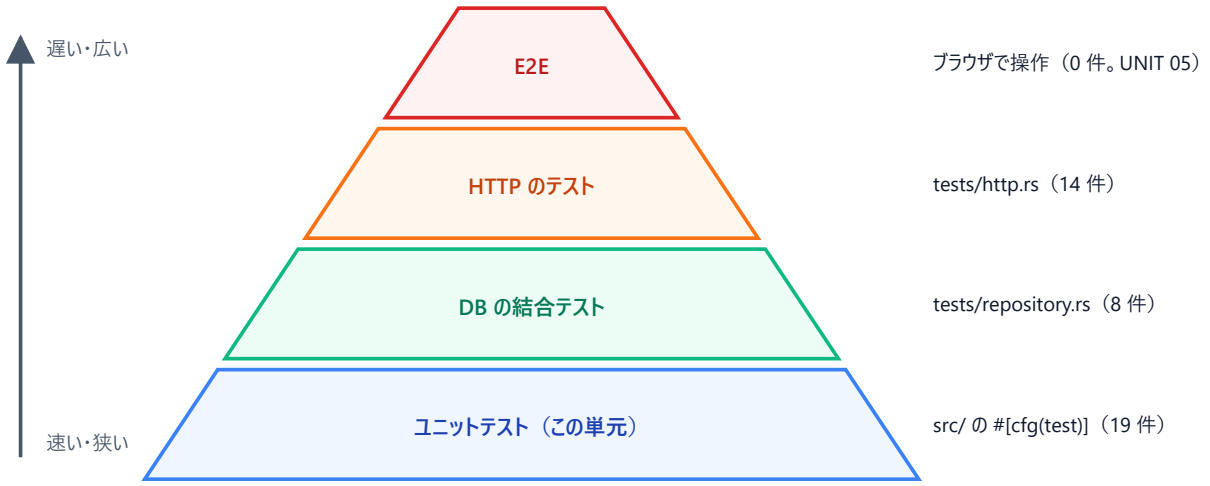


図11-2: テストの段。下の段ほど速く、確かめる範囲が狭いので、たくさん書ける。上の段ほど遅く、広い範囲を一度に確かめるので、大事な流れだけに絞る。右は RustNote にもともと入っているテストの件数

- **ユニットテスト**は、関数1つを確かめます。DB もネットワークも使わないので、数百件あっても1秒もかかりません。決まりの細かい端（80 文字と 81 文字など）は、ここで確かめます。
- 上の段になるほど、本物の DB や HTTP の流れまで含めて確かめられますが、遅く、失敗したときに原因の場所が分かりにくくなります。だから、**細かいことは下の段で、つながりは上の段で**、と役割を分けます。

境界値は、次の図のように選びます。

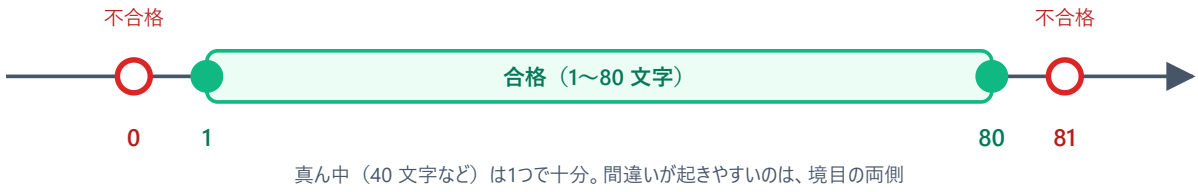


図11-3:「1〜80 文字」の境界値。決まりの端（1 と 80）と、その1つ外側（0 と 81）を必ず試す

- `>` と `>=` の書き間違い、`len()` と `chars().count()` の取り違えのような間違いは、真ん中の値では見つかりません。**端と、その1つ外側**で初めて、答えが変わるからです。

まず動かす最小コード

例1-1: NoteTitle::parse を、表の形で確かめる

> ▼ **このコードがやること（先に日本語で）**：タイトルの決まりを確かめる行（名前・入力・期待する結果）を、表（`Vec<Case>`）に並べます。表のすべての行を `NoteTitle::parse` に渡し、期待どおりなら `ok`、違えば `NG` と、実際の結果を表示します。合わなかった行が1つでもあれば、テストを失敗させます。同じ表を、わざと間違えた版の関数（文字数を `len()` で数える）にも当てて、表が間違いを見つけられるかも確かめます。

```rust
//! 第11巻 UNIT 01: NoteTitle::parse を、境界の値を並べた表で確かめる（表の形のテスト）。
//! 実行: cargo test --test u01_title_table -- --nocapture

use rustnote::domain::NoteTitle;

/// 期待する結果: 合格して入る値か、不合格のときのエラーの文
enum Expect {
    Ok(String),
    Err(&'static str),
}

/// 表の1行: 名前・入力・期待する結果
struct Case {
    label: &'static str,
    input: String,
    expect: Expect,
}

fn ok(label: &'static str, input: impl Into<String>, value: impl Into<String>) -> Case
{
    Case {
        label,
        input: input.into(),
        expect: Expect::Ok(value.into()),
    }
}

fn err(label: &'static str, input: impl Into<String>, message: &'static str) -> Case {
    Case {
        label,
        input: input.into(),
        expect: Expect::Err(message),
    }
}

/// タイトルの決まり（前後の空白を除いて 1〜80 文字）を、境界の値で並べた表
fn title_cases() -> Vec<Case> {
    let a80 = "あ".repeat(80);
    vec![
        ok("ふつうの日本語", "牛乳を買う", "牛乳を買う"),
        ok("前後の空白は取る", "  買い物  ", "買い物"),
        err("空", "", "タイトルを入力してください"),
        err("半角の空白だけ", "   ", "タイトルを入力してください"),
        err("全角の空白だけ", "\u{3000}", "タイトルを入力してください"),
        ok("1文字（下の端）", "あ", "あ"),
        ok("80文字（上の端）", a80.clone(), a80.clone()),
        err(
            "81文字（上の端 + 1）",
            "あ".repeat(81),
```

```
            "タイトルは80文字以内にしてください（現在81文字）",
        ),
        ok("空白を除けば80文字", format!(" {a80} "), a80),
        ok(
            "絵文字の家族（見た目は1つ）",
            "👨\u{200D}👩\u{200D}👧",
            "👨\u{200D}👩\u{200D}👧",
        ),
    ]
}

/// 表のすべての行を `parse` で試し、合わなかった行の説明を返す
fn check(parse: impl Fn(&str) -> Result<String, String>) -> Vec<String> {
    let mut failures = Vec::new();
    for case in title_cases() {
        let actual = parse(&case.input);
        let matched = match (&case.expect, &actual) {
            (Expect::Ok(value), Ok(got)) => value == got,
            (Expect::Err(message), Err(got)) => message == got,
            _ => false,
        };
        let mark = if matched { "ok  " } else { "NG  " };
        let shown = match &actual {
            Ok(got) if got.chars().count() > 10 => {
                format!("Ok（{} 文字）", got.chars().count())
            }
            Ok(got) => format!("Ok({got:?})"),
            Err(message) => format!("Err({message})"),
        };
        println!("{mark}{} → {shown}", case.label);
        if !matched {
            failures.push(case.label.to_string());
        }
    }
    failures
}

/// RustNote の NoteTitle::parse を、表の形にそろえて呼ぶ
fn rustnote_parse(input: &str) -> Result<String, String> {
    NoteTitle::parse(input)
        .map(|title| title.as_str().to_string())
        .map_err(|error| error.to_string())
}

#[test]
fn note_title_follows_the_table() {
    let failures = check(rustnote_parse);
    assert!(failures.is_empty(), "表と合わなかった行: {failures:?}");
}
```

```rust
/// わざと間違えた版: 文字数を len()（バイト数）で数えている
fn broken_parse(input: &str) -> Result<String, String> {
    let trimmed = input.trim();
    let len = trimmed.len();
    if len == 0 {
        return Err("タイトルを入力してください".to_string());
    }
    if len > 80 {
        return Err(format!(
            "タイトルは80文字以内にしてください（現在{len}文字）"
        ));
    }
    Ok(trimmed.to_string())
}

#[test]
fn the_table_catches_a_broken_version() {
    let failures = check(broken_parse);
    println!("わざと間違えた版で合わなかった行: {failures:?}");
    // 表が、間違いを見つけられること（合わない行が1つ以上あること）を確かめる
    assert!(!failures.is_empty(), "表が間違いを見つけられませんでした");
}
```

コピーした `rustnote` フォルダの `tests/u01_title_table.rs` として保存して実行します。

```
cargo test --test u01_title_table -- --nocapture --test-threads=1
```

▼ **実行結果**（最初の1回は `Compiling` の行が続きます。ここでは省いています）

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.87s
     Running tests\u01_title_table.rs (target\debug\deps\u01_title_table-
9a34595b02bbac44.exe)

running 2 tests
test note_title_follows_the_table ... ok  ふつうの日本語 → Ok("牛乳を買う")
ok  前後の空白は取る → Ok("買い物")
ok  空 → Err(タイトルを入力してください)
ok  半角の空白だけ → Err(タイトルを入力してください)
ok  全角の空白だけ → Err(タイトルを入力してください)
ok  1文字（下の端） → Ok("あ")
ok  80文字（上の端） → Ok（80 文字）
ok  81文字（上の端 + 1） → Err(タイトルは80文字以内にしてください（現在81文字）)
ok  空白を除けば80文字 → Ok（80 文字）
ok  絵文字の家族（見た目は1つ） → Ok("👨\u{200d}👩\u{200d}👧")
ok
test the_table_catches_a_broken_version ... ok  ふつうの日本語 → Ok("牛乳を買う")
ok  前後の空白は取る → Ok("買い物")
ok  空 → Err(タイトルを入力してください)
ok  半角の空白だけ → Err(タイトルを入力してください)
ok  全角の空白だけ → Err(タイトルを入力してください)
ok  1文字（下の端） → Ok("あ")
NG  80文字（上の端） → Err(タイトルは80文字以内にしてください（現在240文字）)
NG  81文字（上の端 + 1） → Err(タイトルは80文字以内にしてください（現在243文字）)
NG  空白を除けば80文字 → Err(タイトルは80文字以内にしてください（現在240文字）)
ok  絵文字の家族（見た目は1つ） → Ok("👨\u{200d}👩\u{200d}👧")
わざと間違えた版で合わなかった行: ["80文字（上の端）", "81文字（上の端 + 1）", "空白を除けば80文
字"]
ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.00s
```

▼ コードを1つずつ分解して解説

解説1: 表の1行に、名前を付ける

- `Case` は、表の1行です。`label`（行の名前）・`input`（入力）・`expect`（期待する結果）の3つを持ちます。`ok(...)` と `err(...)` は、行を短く書くための小さな関数です。
- 行の名前に「80文字（上の端）」「81文字（上の端 + 1）」のように、**なぜその行があるのか**を書いておきます。失敗したときに、どの決まりの、どの端で間違えたのかが、名前だけで分かります（エラー1）。
- `"\u{3000}"` は全角の空白、`"\u{200D}"` は、絵文字どうしをつなぐ「見えない文字」（ゼロ幅接合子）です。見えない文字は、ソースに直接書くと区別できないので、`\u{...}` の形で書きます。

解説2: 1つの確かめ方を、全部の行に当てる

- `check` は、表のすべての行に、同じ確かめ方を当てる関数です。比べたい関数を引数（`parse`）で受け取るので、RustNote の `NoteTitle::parse` にも、わざと間違えた版にも、同じ表を使えます。
- 1行ずつ `assert!` で止めずに、合わなかった行の名前を `failures` に集めてから、最後にまとめて `assert!` しています。こうすると、1回の実行で、**合わない行を全部**見つけられます。

解説3: RustNote の版は、全部の行が ok

- 1つ目のテスト（`note_title_follows_the_table`）では、10 行すべてが `ok` でした。半角の空白だけ・全角の空白だけは「タイトルを入力してください」、81 文字は「タイトルは80文字以内にしてください（現在81文字）」です。前後に空白を付けた 82 文字の入力は、空白を除くと 80 文字なので合格でした。
- 家族の絵文字の行は、`Ok("👨\u{200d}👩\u{200d}👧")` と表示されました。`{:?}` で表示すると、見えない文字（ゼロ幅接合子）が `\u{200d}` と書き出されます。

解説4: 間違えた版は、80 と 81 の行で見つかった

- 2つ目のテスト（`the_table_catches_a_broken_version`）では、文字数を `len()`（バイト数）で数える版に、同じ表を当てました。`NG` になったのは、80 文字・81 文字・空白を除けば 80 文字の3行です。日本語の「あ」は1文字3バイトなので、80 文字は 240 バイトと数えられ、「長すぎる」と断られました。
- 一方、「牛乳を買う」（15 バイト）や、家族の絵文字（18 バイト）の行は、間違えた版でも `ok` でした。**短い日本語だけの表では、この間違いを見つけられない**のです。境界値の行を入れておいたから、見つけられました。
- このテストは、「間違えた版で、合わない行が1つ以上あること」を `assert!` で確かめています。表を直していて、うっかり境界値の行を消してしまったら、このテストが失敗して教えてくれます。

もう一歩: 実務に近い形

例1-2: 「長さ」の3つの数え方を比べる

タイトルの文字数は、`chars().count()` で数えています。でも、人が見て「1文字」と思うものと、プログラムの数え方は、いつも同じではありません。

> **▼ このコードがやること（先に日本語で）**: いくつかの文字列について、`chars().count()`（Unicode の文字の数）・`len()`（UTF-8 のバイト数）・`encode_utf16().count()`（UTF-16 の単位の数）の3つを表示し、`NoteTitle::parse` が合格にするかも表示します。

```rust
//! 1つの文字列の「長さ」を、3通りの数え方で比べる。
//! NoteTitle::parse は chars().count()（Unicode の文字の数）で数えている。
//! 実行: cargo run -q --example u01_counts

use rustnote::domain::NoteTitle;

fn main() {
    let samples = [
        ("A", "A"),
        ("あ", "あ"),
        ("が（1文字で表した形）", "\u{304C}"),
        ("が（か + 濁点の2つで表した形）", "\u{304B}\u{3099}"),
        ("👍", "👍"),
        ("家族の絵文字", "👨\u{200D}👩\u{200D}👧"),
        ("ゼロ幅スペースだけ", "\u{200B}"),
    ];
    for (label, text) in samples {
        let parsed = match NoteTitle::parse(text) {
            Ok(_) => "合格".to_string(),
            Err(error) => format!("不合格（{error}）"),
        };
        println!("[{label}]");
        println!(
            "    文字 {} ／ バイト {} ／ UTF-16 {} ／ NoteTitle::parse は {parsed}",
            text.chars().count(),
            text.len(),
            text.encode_utf16().count()
        );
    }
}
```

```
cargo run -q --example u01_counts
```

▼ 実行結果

```
[A]
    文字 1 ／ バイト 1 ／ UTF-16 1 ／ NoteTitle::parse は 合格
[あ]
    文字 1 ／ バイト 3 ／ UTF-16 1 ／ NoteTitle::parse は 合格
[が（1文字で表した形）]
    文字 1 ／ バイト 3 ／ UTF-16 1 ／ NoteTitle::parse は 合格
[が（か + 濁点の2つで表した形）]
    文字 2 ／ バイト 6 ／ UTF-16 2 ／ NoteTitle::parse は 合格
[👍]
    文字 1 ／ バイト 4 ／ UTF-16 2 ／ NoteTitle::parse は 合格
[家族の絵文字]
    文字 5 ／ バイト 18 ／ UTF-16 8 ／ NoteTitle::parse は 合格
[ゼロ幅スペースだけ]
    文字 1 ／ バイト 3 ／ UTF-16 1 ／ NoteTitle::parse は 合格
```



図11-4: 1つに見える家族の絵文字の、3つの数え方。RustNote のタイトルの長さは「文字」で数えるので、この絵文字は 5 文字になる

- **「が」は、2通りの書き方がある:** 1文字で表した「が」（U+304C）は 1 文字ですが、「か」と濁点（U+3099）を組み合わせた「が」は 2 文字です。見た目は同じでも、数は違います。入力の方法（コピーした元の文書など）によって、どちらが来るかは変わります。
- **家族の絵文字は、5 文字:** 見た目は1つですが、👴・つなぐ文字・👵・つなぐ文字・👧の5文字でできています。RustNote では、この絵文字を 16 個入れると、見た目は 16 個でも 80 文字になり、17 個目で断られます。
- **ゼロ幅スペースだけでも合格する:** 見えない文字1つのタイトルは、`NoteTitle::parse` を通ってしまいます。第9巻 UNIT 03 で見つけた、RustNote に残っている改善点です（コードは変えていません）。
- **UTF-16 の数との食い違い:** ブラウザの入力欄の `maxlength`（入力できる長さの上限）は、HTML の決まりでは UTF-16 の単位で数えます。RustNote の作成のフォームは `maxlength=(MAX_TITLE_CHARS)`（80）を付けているので、決まりどおりに数えるブラウザでは、👍 のように UTF-16 で 2 単位の絵文字だけのタイトルは、40 個までしか入りません。サーバーは 80 文字（80 個）まで受け付けるので、ブラウザのほうが先に止めることになります。実際のブラウザでの数え方は、ブラウザを使ったテストで確かめます（UNIT 05（準備中））。どの数え方で「長さ」を決めるかは、仕様として決めて、表のテストに書いておきます。

> **注意:** 人が見て「1文字」と思う単位（書記素クラスタ）で数えるには、標準ライブラリではなく、`unicode-segmentation` のような外のクレートが必要です。RustNote は、SQLite の `length()` と同じ数え方（Unicode の文字の数）にそろえることを優先して、`chars().count()` を使っています（第2巻 UNIT 05・第9巻 UNIT 03）。

よくあるエラーと読み方

この節で作るテストのファイルは、確かめ終わったら必ず消してください。`tests/` に残しておくと、`cargo test` のたびに失敗したり、コンパイルが止まったりします。

エラー1: 表の1行が、期待と合わない

表の「期待する答え」を、まちがえて書いた場合です（`tests/e01_wrong_row.rs`）。

```
use rustnote::domain::NoteTitle;

#[test]
fn title_length_rows() {
    // 表の2行目の「期待する答え」を、まちがえて true（合格）にしてしまった
    let cases = [
        ("80文字", "あ".repeat(80), true),
        ("81文字", "あ".repeat(81), true),
    ];
    for (label, input, should_pass) in cases {
        assert_eq!(
            NoteTitle::parse(&input).is_ok(),
            should_pass,
            "{label} の行が合いません"
        );
    }
}
```

```
cargo test --test e01_wrong_row
```

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 2.63s
     Running tests\e01_wrong_row.rs (target\debug\deps\e01_wrong_row-
ee6ba07e72d8d14e.exe)

running 1 test
test title_length_rows ... FAILED

failures:

---- title_length_rows stdout ----

thread 'title_length_rows' (218780) panicked at tests\e01_wrong_row.rs:11:9:
assertion `left == right` failed: 81文字 の行が合いません
  left: false
 right: true
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace


failures:
    title_length_rows

test result: FAILED. 0 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

error: test failed, to rerun pass `--test e01_wrong_row`
```

- **何が起きたか:** `assertion left == right failed: 81文字 の行が合いません` は、「`assert_eq!` の左（実際の結果）と右（期待）が違いました。81文字 の行です」という意味です。`left: false` は、`NoteTitle::parse` が不合格（`is_ok()` が `false`）を返したこと、`right: true` は、表に「合格のはず」と書いたことです。
- **どちらが間違いか:** テストが失敗したとき、間違っているのは関数のほうとは限りません。今回は、81 文字を合格と書いた**表のほう**が、決まり（1〜80 文字）と違っていました（第9巻 UNIT 01）。
- **名前を付ける理由:** `assert_eq!` の3つ目の引数に `{label}` を入れたので、表のどの行で失敗したかが、すぐ分かりました。名前が無いと、`left: false` と `right: true` だけでは、何の行か分かりません。

エラー2: async fn に #[test] を付けた

DB を使うテストを書くときに、async の関数に、ふつうの `#[test]` を付けてしまった場合です（`tests/e02_async.rs`）。

```rust
use rustnote::domain::NoteTitle;

// async fn に、ふつうの #[test] を付けてしまった
#[test]
async fn title_is_checked() {
    assert!(NoteTitle::parse("買い物").is_ok());
}
```

```
cargo test --test e02_async
```

```
   Compiling rustnote v0.1.0 (...\rustnote)
error: async functions cannot be used for tests
 --> tests\e02_async.rs:5:1
  |
5 |   async fn title_is_checked() {
  |   ^----
  |   |
  |  _`async` because of this
  | |
6 | |     assert!(NoteTitle::parse("買い物").is_ok());
7 | | }
  | |_^

error: could not compile `rustnote` (test "e02_async") due to 1 previous error
```

- **何が起きたか:** 「async の関数は、テストに使えません」というエラーです。ふつうの `#[test]` は、関数を呼ぶだけなので、async の関数の中身（`.await` で待つ部分）を動かせません。
- **直し方:** async の中身が要るテストには、`#[tokio::test]` を付けます。Tokio のランタイムを用意してから、関数を動かしてくれます（第5巻 UNIT 02・第9巻 UNIT 01）。この例の `NoteTitle::parse` は async ではないので、`async` を消して、ふつうの `fn` にするのが正しい直し方です。

エラー3（エラーが出ない間違い）：呼ぶだけで、確かめていないテスト

わざと間違えた版の関数を呼んでいるのに、結果を確かめていないテストです（`tests/e03_no_assert.rs`）。

```rust
/// わざと間違えた版: 文字数を len()（バイト数）で数えている
fn broken_parse(input: &str) -> Result<String, String> {
    let len = input.trim().len();
    if len == 0 || len > 80 {
        return Err("不合格".to_string());
    }
    Ok(input.trim().to_string())
}

#[test]
fn eighty_characters() {
    // 呼ぶだけで、結果を確かめていない
    let _ = broken_parse(&"あ".repeat(80));
}
```

```
cargo test --test e03_no_assert
```

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 1.00s
     Running tests\e03_no_assert.rs (target\debug\deps\e03_no_assert-
18f51359142ae564.exe)

running 1 test
test eighty_characters ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.00s
```

- **何が起きたか:** 間違えた版（80 文字の日本語を「長すぎる」と断る）なのに、テストは `ok` でした。`let _ = ...` で結果を捨てているので、何が返ってきても失敗しません。テストは、**panic したら失敗、しなければ成功**という仕組みだからです。
- **見つけ方:** 例1-1のように、わざと間違えた版を当てて、テストが失敗するかを確かめます。失敗しないテストは、件数を増やすだけで、何も守っていません。この「わざと間違いを入れて、テストが気づくか」を道具で自動にしたものを、ミューテーションテストと呼びます（UNIT 04）。

RustNote ではここで使う

RustNote の `src/domain.rs` の最後には、`#[cfg(test)]` のユニットテストが入っています。タイトルの境界値のテストは、次の形です（抜き出しています）。

```
    #[test]
    fn note_title_counts_characters_not_bytes() {
        // 日本語 80 文字は 240 バイトだが、文字数では 80 なので合格
        let eighty = "あ".repeat(80);
        assert!(NoteTitle::parse(&eighty).is_ok());

        let eighty_one = "あ".repeat(81);
        assert_eq!(
            NoteTitle::parse(&eighty_one),
            Err(TitleError::TooLong { len: 81, max: 80 })
        );
    }
```

- `#[cfg(test)]` の付いた `mod tests` は、`cargo test` のときだけコンパイルされます。本番の実行ファイルには入りません。
- 80 文字と 81 文字の境界を、**日本語で**確かめています。例1-1の「間違えた版」で見たとおり、英数字だけで試していたら、`len()` の取り違えに気づけません。
- 81 文字のときは、`is_err()` で「失敗した」ことだけでなく、`Err(TitleError::TooLong { len: 81, max: 80 })` と、**どう失敗したか**まで比べています。違う理由で失敗している（たとえば「空です」と言われている）間違いも見つけられます。

RustNote の 19 件のユニットテストは、次のコマンドで一覧にできます。`--lib` は「ライブラリの中の `#[cfg(test)]` のテストだけ」という指定です。

```
cargo test --lib -- --list
```

練習問題

問1: `Email::parse` の決まり（前後の空白を除いて小文字にそろえる・`@` の前後が空でない・ドメインに `.` がある・空白が無い・254 バイトまで）を、例1-1と同じ表の形のテストにしてください。境界値として、254 バイトと 255 バイトの行も入れます。

問2: URL の `/notes/{id}` の `{id}` は、`"42".parse::<NoteId>()` で `NoteId` にします。`"1"`・`"0"`・`"-1"`・`"9223372036854775807"`（`i64` の最大値）・`"9223372036854775808"`（最大値 + 1）・`"abc"`・`" 1"`（前に空白）・`"01"` を試して、それぞれどうなるかを表示してください。結果で気になる点はありますか。

問3: `Password::parse` の境界（8〜128 文字）を確かめてください。7・8・128・129 文字のほかに、「空白 7 つ + `a`」と「日本語 8 文字」の行も入れます。パスワードの空白は、タイトルと同じように取り除かれるでしょうか。

解答と解説

問1の解答:

`tests/u01_q1.rs` です。例1-1の `Case` を使わずに、タプル（名前, 入力, 期待）の `Vec` で、もっと短く書いています。

```rust
//! 問1の解答: Email::parse を、境界と「よくある書き間違い」を並べた表で確かめる。
//! 実行: cargo test --test u01_q1 -- --nocapture

use rustnote::domain::Email;

#[test]
fn email_follows_the_table() {
    let long_ok = format!("{}@example.com", "a".repeat(242));
    let long_ng = format!("{}@example.com", "a".repeat(243));
    //  (名前, 入力, 期待: Ok なら そろえた値、Err なら エラーの文)
    let cases: Vec<(&str, String, Result<String, &str>)> = vec![
        (
            "ふつう",
            "alice@example.com".into(),
            Ok("alice@example.com".into()),
        ),
        (
            "空白と大文字",
            "  Alice@Example.COM  ".into(),
            Ok("alice@example.com".into()),
        ),
        (
            "@ が無い",
            "alice".into(),
            Err("メールアドレスに @ がありません"),
        ),
        (
            "@ の前が空",
            "@example.com".into(),
            Err("メールアドレスの形式が正しくありません"),
        ),
        (
            "@ の後ろが空",
            "alice@".into(),
            Err("メールアドレスの形式が正しくありません"),
        ),
        (
            "ドメインに . が無い",
            "alice@example".into(),
            Err("メールアドレスの形式が正しくありません"),
        ),
        (
            "ドメインが . で始まる",
            "alice@.example.com".into(),
            Err("メールアドレスの形式が正しくありません"),
        ),
        (
            "途中に空白",
            "al ice@example.com".into(),
```

```
            Err("メールアドレスの形式が正しくありません"),
        ),
        (
            "@ が2つ",
            "a@b@example.com".into(),
            Err("メールアドレスの形式が正しくありません"),
        ),
        ("254 バイト（上の端）", long_ok.clone(), Ok(long_ok)),
        (
            "255 バイト（上の端 + 1）",
            long_ng,
            Err("メールアドレスが長すぎます"),
        ),
    ];
    let mut failures = Vec::new();
    for (label, input, expected) in cases {
        let actual = Email::parse(&input)
            .map(|email| email.as_str().to_string())
            .map_err(|error| error.to_string());
        let matched = match (&expected, &actual) {
            (Ok(value), Ok(got)) => value == got,
            (Err(message), Err(got)) => message == got,
            _ => false,
        };
        let shown = match &actual {
            Ok(got) if got.len() > 30 => format!("Ok（{} バイト）", got.len()),
            Ok(got) => format!("Ok({got:?})"),
            Err(message) => format!("Err({message})"),
        };
        println!("{} {label} → {shown}", if matched { "ok" } else { "NG" });
        if !matched {
            failures.push(label);
        }
    }
    assert!(failures.is_empty(), "表と合わなかった行: {failures:?}");
}
```

```
cargo test --test u01_q1 -- --nocapture
```

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.65s
     Running tests\u01_q1.rs (target\debug\deps\u01_q1-bce08f7bd283041f.exe)

running 1 test
ok ふつう → Ok("alice@example.com")
ok 空白と大文字 → Ok("alice@example.com")
ok @ が無い → Err(メールアドレスに @ がありません)
ok @ の前が空 → Err(メールアドレスの形式が正しくありません)
ok @ の後ろが空 → Err(メールアドレスの形式が正しくありません)
ok ドメインに . が無い → Err(メールアドレスの形式が正しくありません)
ok ドメインが . で始まる → Err(メールアドレスの形式が正しくありません)
ok 途中に空白 → Err(メールアドレスの形式が正しくありません)
ok @ が2つ → Err(メールアドレスの形式が正しくありません)
ok 254 バイト（上の端） → Ok（254 バイト）
ok 255 バイト（上の端 + 1） → Err(メールアドレスが長すぎます)
test email_follows_the_table ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.00s
```

- 11 行すべてが通りました。`" Alice@Example.COM "` は、空白が取られ、小文字の `alice@example.com` にそろいました（第9巻 UNIT 03）。
- 254 バイトの行は、`a` を 242 個 + `@example.com`（12 バイト）で作っています。上限はバイト数（`normalized.len()`）で決めているので、境界値もバイト数で作ります。決まりがどの数え方かによって、境界値の作り方も変わります。
- `"a@b@example.com"` は、最初の `@` で分けると、ドメインが `b@example.com` になり、その中に `@` があるので断られました。

問2の解答:

`tests/u01_q2.rs` です。

```rust
//! 問2の解答: URL の /notes/{id} の {id} を NoteId にするときの境界を確かめる。
//! 実行: cargo test --test u01_q2 -- --nocapture

use rustnote::domain::NoteId;

#[test]
fn note_id_boundaries() {
    let inputs = [
        "1",
        "0",
        "-1",
        "9223372036854775807",
        "9223372036854775808",
        "abc",
        " 1",
        "01",
    ];
    for input in inputs {
        match input.parse::<NoteId>() {
            Ok(id) => println!("{input:?} → Ok({})", id.get()),
            Err(error) => println!("{input:?} → Err({error})"),
        }
    }
    // 下の端と上の端を、assert で固定する
    assert!("0".parse::<NoteId>().is_err());
    assert!("1".parse::<NoteId>().is_ok());
    assert!("9223372036854775807".parse::<NoteId>().is_ok());
    assert!("9223372036854775808".parse::<NoteId>().is_err());
}
```

```
cargo test --test u01_q2 -- --nocapture
```

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.61s
     Running tests\u01_q2.rs (target\debug\deps\u01_q2-07596b0af0a888f4.exe)

running 1 test
"1" → Ok(1)
"0" → Err(ID は 1 以上の数です)
"-1" → Err(ID は 1 以上の数です)
"9223372036854775807" → Ok(9223372036854775807)
"9223372036854775808" → Err(ID は数字で指定してください)
"abc" → Err(ID は数字で指定してください)
" 1" → Err(ID は数字で指定してください)
"01" → Ok(1)
test note_id_boundaries ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.00s
```

- 下の端は、`"1"` が合格、`"0"` と `"-1"` が「ID は 1 以上の数です」でした。上の端は、`i64` の最大値 `9223372036854775807` が合格、1 つ大きい `9223372036854775808` が不合格です。
- **気になる点1:** `9223372036854775808` は数なのに、「ID は数字で指定してください」と言われました。`NoteId` は、まず `i64` として読み、読めなければ「数字ではない」としているからです（`src/domain.rs` の `from_str`）。大きすぎる数も「数字ではない」になります。URL の ID は利用者が手で打つものではないので大きな問題ではありませんが、エラーの文が正確でないことは、テストを書いて初めて分かります。
- **気になる点2:** `"01"` は `1` として合格しました。つまり `/notes/01` と `/notes/1` は、同じノートを開きます。困ることはありませんが、「同じものを指す URL が2つある」ことは、知っておくと、ログを調べるときに迷いません。
- `" 1"`（前に空白）は不合格でした。`i64` の `parse` は、空白を取り除かないからです。タイトルの `trim()` と比べると、型ごとに空白の扱いが違うことが分かります。

問3の解答:

`tests/u01_q3.rs` です。

```rust
//! 問3の解答: Password::parse の境界（8〜128 文字）と、空白の扱いを確かめる。
//! 実行: cargo test --test u01_q3 -- --nocapture

use rustnote::domain::Password;

#[test]
fn password_boundaries() {
    let cases = [
        ("7文字（下の端 − 1）", "a".repeat(7), false),
        ("8文字（下の端）", "a".repeat(8), true),
        ("128文字（上の端）", "a".repeat(128), true),
        ("129文字（上の端 + 1）", "a".repeat(129), false),
        ("空白7つ + a（8文字）", format!("{}a", " ".repeat(7)), true),
        ("日本語8文字（24 バイト）", "あ".repeat(8), true),
    ];
    for (label, input, should_pass) in cases {
        let result = Password::parse(&input);
        match &result {
            Ok(password) => println!("{label} → Ok({password:?})"),
            Err(error) => println!("{label} → Err({error})"),
        }
        assert_eq!(result.is_ok(), should_pass, "{label} の行が合いません");
    }
}
```

```
cargo test --test u01_q3 -- --nocapture
```

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.60s
     Running tests\u01_q3.rs (target\debug\deps\u01_q3-ead5ae5e7277cf12.exe)

running 1 test
7文字（下の端 − 1） → Err(パスワードは8文字以上にしてください)
8文字（下の端） → Ok(Password(***))
128文字（上の端） → Ok(Password(***))
129文字（上の端 + 1） → Err(パスワードは128文字以内にしてください)
空白7つ + a（8文字） → Ok(Password(***))
日本語8文字（24 バイト） → Ok(Password(***))
test password_boundaries ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.00s
```

- 7 文字は「パスワードは8文字以上にしてください」、129 文字は「パスワードは128文字以内にしてください」、8 文字と 128 文字は合格です。合格の行は `Password(***)` と表示され、中身は出ません（第9巻 UNIT 03）。

- 「空白 7 つ + `a`」も 8 文字として合格しました。パスワードは、タイトルと違って `trim()` しません。`src/domain.rs` のコメントに「パスワードは空白も含めて利用者が決めたものなので、trim しない」とあります。空白をこっそり取ると、利用者が決めたパスワードと違うものが保存されてしまうからです。
- 「日本語 8 文字」（24 バイト）も合格でした。パスワードの長さも `chars().count()`（文字の数）で数えています。

まとめ

- テストには段がある。ユニットテストは、関数1つを速く確かめる段で、決まりの細かい端はここで確かめる。上の段（DB・HTTP・E2E）は、つながりを確かめる段で、大事な流れに絞る。
- 境界値（端と、その1つ外側）と、間違えやすい入力（空白・全角の空白・日本語・絵文字）を、名前付きの行で表に並べる。1つの確かめ方を全部の行に当て、合わない行をまとめて報告する。
- わざと間違えた版に同じ表を当てて、テストが失敗することを確かめる。結果を確かめないテストは、何も守っていない。

UNIT 02: リポジトリの結合テスト

この単元のゴール

- リポジトリ（`NoteRepository` など）を、偽物ではなく**本物の DB**（一時フォルダの libSQL ファイル）で確かめる理由を、SQL の細かい決まりの例で説明できる
- 準備（Arrange）・操作（Act）・確かめ（Assert）の形で、成功・見つからない・他人・版の食い違いを、1つずつ確かめるテストを書ける
- わざと SQL を壊したとき、ユニットテストは通ったまま、結合テストだけが気づく様子を確かめられる
- テストどうしが DB ファイルを使い回したときに起きる、実行のたびに変わる失敗を読み、テストごとに新しい DB を作る形に直せる

たとえ話でつかむ

レストランの厨房で、新しいスープの味を確かめる場面を考えます。

- UNIT 01 のユニットテストは、**材料1つずつの味見**です。「塩はこの量」「だしはこの濃さ」と、1つずつ確かめます。速くて、どこが悪いかもすぐ分かります。
- ところが、材料が全部正しくても、**実際の鍋で煮る**と、火加減や、入れる順番のせいで、思った味にならないことがあります。そこで、本物の鍋で煮て味見をします。これが、この単元の**結合テスト**です。
- 味見のたびに、**洗った新しい鍋**を使います。前の人が使った鍋のままだと、前のスープの味が残っていて、どちらの味なのか分からなくなります。2人が同時に1つの鍋を使えば、もっと混乱します。

厨房	RustNote のテスト
材料1つずつの味見	ユニットテスト（`escape_like` や `NoteTitle::parse` を、DB なしで確かめる。UNIT 01）
本物の鍋で煮て味見	リポジトリの結合テスト（本物の libSQL に、本物の SQL を流す）
火加減・入れる順番	SQL の細かい決まり（`LIKE` の `ESCAPE`・`WHERE` の条件・`UNIQUE`・`ON DELETE CASCADE`）
洗った新しい鍋	テストごとに作る一時フォルダの DB（`common::test_db()`）
前のスープの味が残る	前のテストのデータが残った DB。結果が実行のたびに変わる

> **用語:** 結合テスト（けつごうテスト：いくつかの部品を、本物どうしでつないで確かめるテスト。ここでは、リポジトリと本物の DB をつなぐ）

> **用語:** Arrange・Act・Assert（アレンジ・アクト・アサート：テストを「準備する」「操作する」「確かめる」の3つに分けて書く形）

図で理解する

テストの段と、それぞれが確かめるものです。この単元は、下から2段目です。

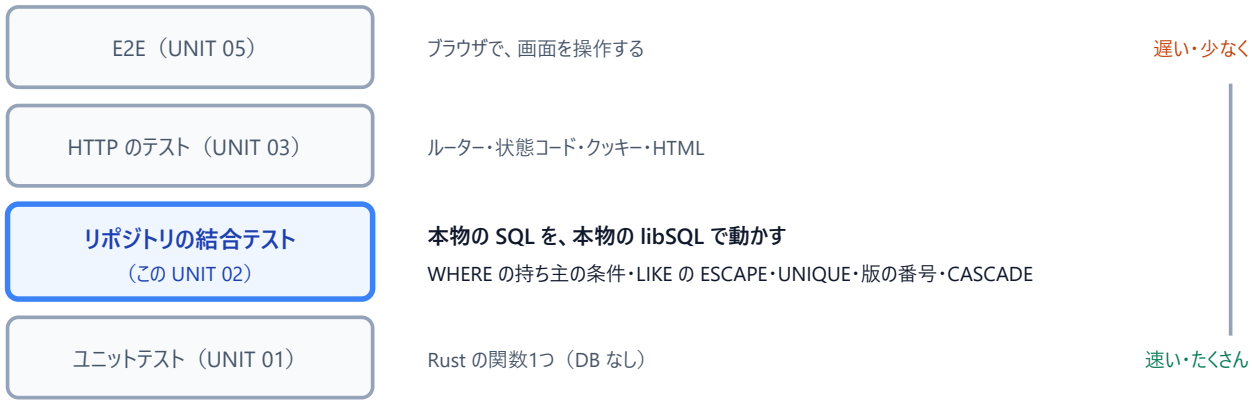


図11-5: テストの段。リポジトリの結合テストは、ユニットテストでは確かめられない「SQL そのもの」を、本物の DB で確かめる

テストごとに、新しい DB を用意します。



図11-6: テストごとの DB。RustNote の common::test_db() は、テストごとに一時フォルダを作り、マイグレーションを当てた空の DB を渡す

まず動かす最小コード

例2-1: NoteRepository の約束を、1つずつ確かめる

`NoteRepository` の6つのメソッドには、それぞれ約束があります（`src/repository/notes.rs` のコメント）。たとえば「他人のノート・存在しないノートは None」「`expected_version` が今の version と違えば `VersionMismatch`」です。これを、本物の DB で1つずつ確かめます。

> **▼ このコードがやること（先に日本語で）**：空の DB に alice と bob を登録します（準備）。alice のノートを作り、探す・更新する・完了にする・消す、を、alice と bob の両方から試します（操作）。それぞれの結果を、1行ずつ表示します（確かめ）。ここでは結果を見るために表示していますが、本物のテストでは `assert!` で確かめます。

```rust
//! 第11巻 UNIT 02: NoteRepository の約束を、本物の DB（一時フォルダの libSQL ファイル）で1つずつ確かめる。
//! 実行: cargo test --test u02_contract -- --nocapture

mod common;

use rustnote::{
    domain::{Email, NewNote, Note, NoteId},
    error::RepoError,
    repository::{LibsqlNoteRepository, LibsqlUserRepository, NoteRepository, UserRepository},
};

/// ノートの大事な所だけを、1行の文字列にする
fn brief(result: Result<Note, RepoError>) -> String {
    match result {
        Ok(note) => format!(
            "Ok (id={} version={} done={} title={:?}) ",
            note.id, note.version, note.done, note.title
        ),
        Err(error) => format!("Err({error:?})"),
    }
}

fn new_note(title: &str) -> NewNote {
    NewNote::parse(title, "").expect("テスト用のノート")
}

#[tokio::test]
async fn note_repository_contract() {
    // Arrange: 空の DB と、2人のユーザー
    let test = common::test_db().await;
    let users = LibsqlUserRepository::new(test.db.clone());
    let notes = LibsqlNoteRepository::new(test.db.clone());
    let alice = users
        .create(
            &Email::parse("alice@example.com").unwrap(),
            "$argon2id$dummy",
        )
        .await
        .unwrap();
    let bob = users
        .create(&Email::parse("bob@example.com").unwrap(), "$argon2id$dummy")
        .await
        .unwrap();

    // Act と Assert: 約束を1つずつ
    let created = notes.insert(alice.id, &new_note("初版")).await.unwrap();
    let id = created.id;
```

```
    println!(
        "insert (alice)              → {}",
        brief(Ok(created.clone()))
    );
    println!(
        "   作成と更新の日時が同じ    → {}",
        created.created_at == created.updated_at
    );

    let found = notes.find(alice.id, id).await.unwrap();
    println!(
        "find (alice)                → 見つかる: {}",
        found.is_some()
    );
    let found = notes.find(bob.id, id).await.unwrap();
    println!(
        "find (bob)                  → 見つかる: {}",
        found.is_some()
    );

    let result = notes.update(alice.id, id, &new_note("第2版"), 1).await;
    println!("update (alice, version 1)   → {}", brief(result));
    let result = notes.update(alice.id, id, &new_note("古い画面"), 1).await;
    println!("update (alice, version 1)   → {}", brief(result));
    let missing = NoteId::new(99).unwrap();
    let result = notes.update(alice.id, missing, &new_note("無い"), 1).await;
    println!("update (alice, id 99)       → {}", brief(result));
    let result = notes.update(bob.id, id, &new_note("乗っ取り"), 2).await;
    println!("update (bob, version 2)     → {}", brief(result));

    let result = notes.toggle_done(alice.id, id).await;
    println!("toggle_done (alice)         → {}", brief(result));

    println!(
        "delete (bob)                → {:?}",
        notes.delete(bob.id, id).await
    );
    println!(
        "delete (alice)              → {:?}",
        notes.delete(alice.id, id).await
    );
    println!(
        "delete (alice, もう一度)    → {:?}",
        notes.delete(alice.id, id).await
    );
}
```

コピーした `rustnote` フォルダの `tests/u02_contract.rs` として保存して実行します。

```
cargo test --test u02_contract -- --nocapture
```

▼ 実行結果

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.62s
     Running tests\u02_contract.rs (target\debug\deps\u02_contract-
28783a9a9988e5a0.exe)

running 1 test
insert (alice)              → Ok (id=1 version=1 done=false title="初版")
   作成と更新の日時が同じ    → true
find (alice)                → 見つかる: true
find (bob)                  → 見つかる: false
update (alice, version 1)   → Ok (id=1 version=2 done=false title="第2版")
update (alice, version 1)   → Err(VersionMismatch)
update (alice, id 99)       → Err(NotFound)
update (bob, version 2)     → Err(NotFound)
toggle_done (alice)         → Ok (id=1 version=3 done=true title="第2版")
delete (bob)                → Err(NotFound)
delete (alice)              → Ok(())
delete (alice, もう一度)    → Err(NotFound)
test note_repository_contract ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.07s
```

▼ コードを1つずつ分解して解説

解説1: 準備は、毎回まっさらな DB から

- `common::test_db().await` が、新しい一時フォルダに DB を作り、マイグレーション（表を作る SQL。第13巻 UNIT 02）を当てて渡します。だから、最初のノートの id は、いつも 1 です。
- ユーザーは `LibsqlUserRepository::create` で登録します。パスワードのハッシュは、ここでは関係ないので `"$argon2id$dummy"` という仮の文字列です。

解説2: 成功だけでなく、断られる場合を全部試す

- `update` は4回呼びました。正しい version（1）なら成功して version 2 に、同じ version 1 でもう一度なら `VersionMismatch`（版の食い違い。別のタブで先に更新された）、無い id 99 なら `NotFound`、bob からなら `NotFound` です。
- bob からの更新は、version を正しい 2 にしても `NotFound` でした。「他人のノートは、あっても無いのと同じ」という約束です（第8巻 UNIT 07）。`VersionMismatch` にすると、「そのノートはある」と bob に教えてしまいます。
- 削除は、bob から → `NotFound`、alice から → `Ok(())`、もう一度 → `NotFound` です。1回目と3回目の `NotFound` は、理由は違います（他人／もう無い）が、外から見ると同じです。

解説3: 本物の DB でも、テストは速い

- 実行結果の最後の行のとおり、このテストは数十ミリ秒で終わります。一時フォルダのファイルなので、ネットワークの待ち時間がありません。RustNote の `tests/repository.rs` の8つのテストも、まとめて 1 秒かかりません。
- 速いので、「本物の DB は遅いから偽物にする」必要は、ここではありません。

もう一歩: 実務に近い形

例2-2: SQL を1か所壊すと、どのテストが気づくか

本物の DB で確かめると、何が良いのでしょうか。SQL をわざと1か所壊して、確かめます。試すときは、練習用のコピーの `src/repository/notes.rs` を書き換え、確かめたら必ず元に戻してください。

検索の SQL の、`ESCAPE '\'`（`\` のうしろの文字を、ただの文字として扱う、という指定）を消します。

```
        // 元: "user_id = ?1 AND (title LIKE ?2 ESCAPE '\\' OR body LIKE ?2 ESCAPE
'\\')"
        let condition = "user_id = ?1 AND (title LIKE ?2 OR body LIKE ?2)";
```

まず、`escape_like` のユニットテスト（`src/repository/notes.rs` の中の `like_special_characters_are_escaped`）を動かします。

```
cargo test --lib like_special
```

```
running 1 test
test repository::notes::tests::like_special_characters_are_escaped ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 18 filtered out; finished
in 0.00s
```

ユニットテストは**通りました**。次に、リポジトリの結合テストを動かします。

```
cargo test --test repository
```

```
running 8 tests
test migrations_are_applied_only_once ... ok
test duplicate_email_is_a_unique_violation ... ok
test update_uses_optimistic_locking ... ok
test deleting_a_session_logs_out ... ok
test sessions_expire ... ok
test notes_are_isolated_per_owner ... ok
test toggle_and_delete ... ok
test search_filters_by_keyword_and_paginates ... FAILED

failures:

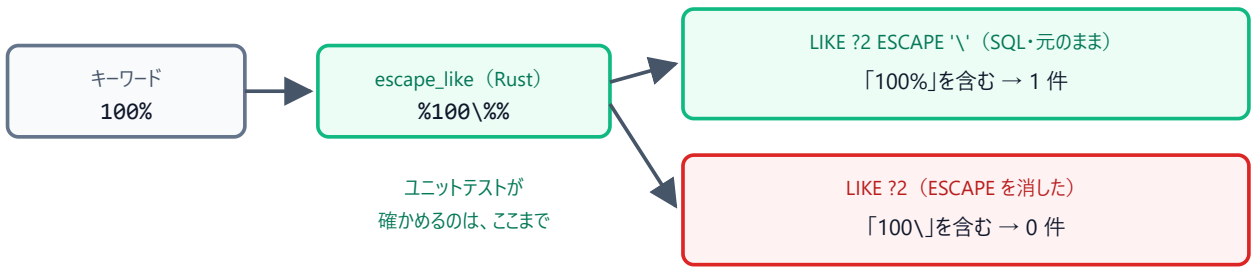
---- search_filters_by_keyword_and_paginates stdout ----

thread 'search_filters_by_keyword_and_paginates' (215108) panicked at
tests\repository.rs:198:5:
assertion `left == right` failed
  left: 0
 right: 1
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace


failures:
    search_filters_by_keyword_and_paginates

test result: FAILED. 7 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.18s

error: test failed, to rerun pass `--test repository`
```



作った模様（Rust）が正しくても、SQL がその模様をどう読むかは、本物の DB でしか確かめられない

図11-7: ESCAPE を消したときの違い。escape_like の結果は同じでも、SQL が \ を「ただの \」として読むので、何も見つからなくなる

- **何が起きたか:** 8つのテストのうち、`search_filters_by_keyword_and_paginates` だけが失敗しました。198行目は `assert_eq!(percent.total_count, 1);` で、「100%」で検索した件数が、1 のはずが 0 でした。

- **なぜ 0 件か:** `escape_like` は、`100%` を `100\%` に書き換えます（これは正しいままです）。ところが SQL に `ESCAPE '\'` が無いと、`\` は「次の文字をただの文字にする印」ではなく、**ただの** `\` として読まれます。だから「`100\` を含むノート」を探してしまい、1件も見つかりません。
- **学べること:** Rust の関数（`escape_like`）と、SQL（`ESCAPE '\'`）は、**2つそろって**初めて正しく動きます。ユニットテストは片方しか見ていないので、もう片方が壊れても気づけません。偽物のリポジトリ（第4巻 UNIT 02）でも、SQL は動かないので気づけません。**SQL を書いたら、本物の DB で動かすテストを置く**のが、この単元の中心です。確かめたら、`src/repository/notes.rs` を元に戻します。

よくあるエラーと読み方

エラー1: `assertion failed: matches!(…)` だけで、何が返ったか分からない

今度は、削除の SQL から、持ち主の条件 `AND user_id = ?2` を消してみます（練習用のコピーで試し、必ず元に戻します）。

```
                "DELETE FROM notes WHERE id = ?1",
```

```
cargo test --test repository
```

```
running 8 tests
test migrations_are_applied_only_once ... ok
test duplicate_email_is_a_unique_violation ... ok
test deleting_a_session_logs_out ... ok
test update_uses_optimistic_locking ... ok
test sessions_expire ... ok
test toggle_and_delete ... ok
test notes_are_isolated_per_owner ... FAILED
test search_filters_by_keyword_and_paginates ... ok

failures:

---- notes_are_isolated_per_owner stdout ----

thread 'notes_are_isolated_per_owner' (218972) panicked at tests\repository.rs:90:5:
assertion failed: matches!(notes.delete(bob.id, note.id).await,
Err(RepoError::NotFound))
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace


failures:
    notes_are_isolated_per_owner

test result: FAILED. 7 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.19s

error: test failed, to rerun pass `--test repository`
```

- **何が起きたか:** `notes_are_isolated_per_owner` が、90 行目で失敗しました。bob が alice のノートを消そうとしたら `NotFound` になるはずが、そうならなかった、という意味です。テストが無ければ、**誰でも、他人のノートを id だけで消せる**ところでした（IDOR。第8巻 UNIT 01）。
- **気づきにくい点:** SQL の `?2` を消したのに、`params![id, owner]` で2つの値を渡したままでも、エラーにはなりませんでした。libsql は、使われない値を黙って無視しました。**コンパイルも、実行も通る**ので、テストだけが頼りです。
- **困る点:** `assertion failed: matches!(…)` は、**実際に何が返ったか**を表示しません。`Ok(())` だったのか、別のエラーだったのか、この表示だけでは分かりません。

直し方: `assert!` の2つ目の引数に、失敗したときの文と、実際の値を書いておきます（`tests/u02_owner_delete.rs`）。

```rust
//! 第11巻 UNIT 02 のエラー1の直し方: 失敗したときに「実際の値」が出る形で確かめる。
//! 実行: cargo test --test u02_owner_delete

mod common;

use rustnote::{
    domain::{Email, NewNote},
    error::RepoError,
    repository::{LibsqlNoteRepository, LibsqlUserRepository, NoteRepository,
UserRepository},
};

#[tokio::test]
async fn other_user_cannot_delete() {
    let test = common::test_db().await;
    let users = LibsqlUserRepository::new(test.db.clone());
    let notes = LibsqlNoteRepository::new(test.db.clone());
    let alice = users
        .create(
            &Email::parse("alice@example.com").unwrap(),
            "$argon2id$dummy",
        )
        .await
        .unwrap();
    let bob = users
        .create(&Email::parse("bob@example.com").unwrap(), "$argon2id$dummy")
        .await
        .unwrap();
    let note = notes
        .insert(alice.id, &NewNote::parse("買い物", "").unwrap())
        .await
        .unwrap();

    let result = notes.delete(bob.id, note.id).await;
    // 2つ目の引数に、失敗したときに出す文と、実際の値を書いておく
    assert!(
        matches!(result, Err(RepoError::NotFound)),
        "bob の削除は NotFound のはずが、{result:?} だった"
    );
    let still_there = notes.find(alice.id, note.id).await.unwrap();
    assert!(still_there.is_some(), "alice のノートが消えている");
}
```

壊したままの SQL で動かすと、次のように表示されます。

```
cargo test --test u02_owner_delete
```

```
running 1 test
test other_user_cannot_delete ... FAILED

failures:

---- other_user_cannot_delete stdout ----

thread 'other_user_cannot_delete' (215892) panicked at tests\u02_owner_delete.rs:35:5:
bob の削除は NotFound のはずが、Ok(()) だった
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace


failures:
    other_user_cannot_delete

test result: FAILED. 0 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.05s

error: test failed, to rerun pass `--test u02_owner_delete`
```

- 「bob の削除は NotFound のはずが、`Ok(())` だった」と、実際の値が出ました。「bob の削除が**成功してしまった**」と、すぐに分かります。
- 元の SQL に戻すと、このテストは通ります（`test other_user_cannot_delete ... ok`）。**失敗したときに読む人**のために、確かめる所に一言書いておくのが、良いテストです。

エラー2: テストが、同じ DB ファイルを使い回した

テストごとに DB を作るのが面倒で、決まった1つのファイル（`rustnote-shared-test.db`）を、どのテストからも開くようにした場合です（`tests/u02_shared.rs`）。2つのテストは、どちらも alice を登録します。

```rust
//! 第11巻 UNIT 02 のエラー2: 2つのテストが、同じ DB ファイルを使い回したとき。
//! 実行: cargo test --test u02_shared

use rustnote::{
    db::{self, Db},
    domain::Email,
    repository::{LibsqlUserRepository, UserRepository},
};

/// どのテストも、決まった1つのファイルを開いてしまう（一時フォルダを作らない）
async fn shared_db() -> Db {
    let path = std::env::temp_dir().join("rustnote-shared-test.db");
    let db = Db::open_local(path).await.expect("DB を開けません");
    db::migrate(&db).await.expect("マイグレーションに失敗");
    db
}

#[tokio::test]
async fn sign_up_alice() {
    let users = LibsqlUserRepository::new(shared_db().await);
    let email = Email::parse("alice@example.com").unwrap();
    users.create(&email, "$argon2id$dummy").await.unwrap();
}

#[tokio::test]
async fn alice_can_be_found() {
    let users = LibsqlUserRepository::new(shared_db().await);
    let email = Email::parse("alice@example.com").unwrap();
    users.create(&email, "$argon2id$dummy").await.unwrap();
    assert!(users.find_by_email(&email).await.unwrap().is_some());
}
```

ファイルが無い状態から、1回目を動かします。

```
cargo test --test u02_shared
```

```
running 2 tests
test sign_up_alice ... ok
test alice_can_be_found ... FAILED

failures:

---- alice_can_be_found stdout ----

thread 'alice_can_be_found' (219836) panicked at tests\u02_shared.rs:29:51:
called `Result::unwrap()` on an `Err` value: UniqueViolation("SQLite failure: `UNIQUE
constraint failed: users.email`")
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace


failures:
    alice_can_be_found

test result: FAILED. 1 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.04s

error: test failed, to rerun pass `--test u02_shared`
```

- **何が起きたか:** `sign_up_alice` は通り、`alice_can_be_found` が `UniqueViolation`（同じメールアドレスが、もうある）で失敗しました。2つのテストは**同時に**動くので、先に alice を登録したほうが通り、あとのほうが失敗します。どちらが先かは、その時しだいです。

同じことを、ファイルを消してから 20 回くり返すと、毎回1つは通り、もう1つの失敗の理由は、次のように変わりました。

失敗の理由	回数（20 回のうち）	どこで止まったか
`UniqueViolation`（alice がもういる）	17 回	alice を登録するところ
`database is locked`（ほかの接続が書いている最中）	3 回	DB を開くところ

```
thread 'alice_can_be_found' (215524) panicked at tests\u02_shared.rs:13:41:
DB を開けません: SqliteFailure(5, "database is locked")
```

- この 20 回では出ませんでしたが、前の版（Topcoat 0.10・Rust 1.99.0）で8回試したときには、マイグレーションのところで `table notes already exists`（表がもうある）で止まったことも1回ありました。
- 2つのテストが、同じファイルを**同時に**開いて、同時にマイグレーションを当てようとするので、「表を作る」「書き込む」がぶつかります。どこでぶつかるかは、毎回違います。

ファイルを消さずに、もう一度動かします。

```
running 2 tests
test sign_up_alice ... FAILED
test alice_can_be_found ... FAILED

failures:

---- sign_up_alice stdout ----

thread 'sign_up_alice' (218360) panicked at tests\u02_shared.rs:22:51:
called `Result::unwrap()` on an `Err` value: UniqueViolation("SQLite failure: `UNIQUE
constraint failed: users.email`")
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace

---- alice_can_be_found stdout ----

thread 'alice_can_be_found' (218476) panicked at tests\u02_shared.rs:29:51:
called `Result::unwrap()` on an `Err` value: UniqueViolation("SQLite failure: `UNIQUE
constraint failed: users.email`")


failures:
    alice_can_be_found
    sign_up_alice

test result: FAILED. 0 passed; 2 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.03s

error: test failed, to rerun pass `--test u02_shared`
```

- 今度は、**両方とも**失敗しました。前の実行で登録した alice が、ファイルに残っているからです。
- **直し方**: RustNote のテストのように、`common::test_db()` で、テストごとに新しい一時フォルダの DB を作ります。テストの結果が「そのテストの中でしたこと」だけで決まるようにすると、同時に動かしても、何回動かしても、同じ結果になります。確かめたら、`tests/u02_shared.rs` と、一時フォルダ（PowerShell なら `$env:TEMP`）にできた `rustnote-shared-test.db` で始まるファイルを消しておきます。
- **学べること**: 「ときどき失敗するテスト」は、テストどうしが何かを**共有している**ことが多い原因です。DB のファイル・決まったポート番号・環境変数・今の時刻などです。失敗したら、まず「このテストは、ほかのテストと何を共有しているか」を探します。

RustNote ではここで使う

テストごとの DB は、`tests/common/mod.rs` の `test_db` が作ります（抜き出しています）。

```rust
/// 一時フォルダの DB。TempDir が消えるとフォルダごと削除される
pub struct TestDb {
    pub db: Db,
    _dir: TempDir,
}

/// マイグレーション済みの空の DB を作る
pub async fn test_db() -> TestDb {
    let dir = tempfile::tempdir().expect("一時フォルダを作れません");
    let db = Db::open_local(dir.path().join("test.db"))
        .await
        .expect("DB を開けません");
    db::migrate(&db).await.expect("マイグレーションに失敗");
    TestDb { db, _dir: dir }
}
```

- `tempfile::tempdir()` は、ほかと重ならない名前の一時フォルダを作ります。テストが同時にいくつ動いても、それぞれ別のフォルダです。
- `TestDb` は、`db` と一緒に `TempDir` を持っています。テストが終わって `TestDb` が片付けられる（drop される）と、`TempDir` もフォルダごと消えます（第3巻 UNIT 01）。項目の名前が `_dir` と `_` で始まるのは、「使わないが、持っておくこと自体に意味がある」という印です。`TempDir` を持っておくのは、テストの間はフォルダを残し、終わったら消すためです。
- 本番と同じ `db::migrate` を当てるので、テストの DB の表は、いつも本番と同じ形です。マイグレーションを足せば、テストの DB にも自動で入ります。

`tests/repository.rs` は、この `test_db` を使って、リポジトリの約束を8つのテストで確かめています。ユーザーを登録する手順は、何度も出てくるので、小さな関数にまとめています（抜き出しています）。

```rust
/// テスト用のユーザーを登録する
async fn create_user(users: &LibsqlUserRepository, email: &str) -> User {
    let email = Email::parse(email).unwrap();
    users.create(&email, "$argon2id$dummy").await.unwrap()
}
```

```
cargo test --test repository
```

```
running 8 tests
test migrations_are_applied_only_once ... ok
test duplicate_email_is_a_unique_violation ... ok
test update_uses_optimistic_locking ... ok
test deleting_a_session_logs_out ... ok
test sessions_expire ... ok
test notes_are_isolated_per_owner ... ok
test toggle_and_delete ... ok
test search_filters_by_keyword_and_paginates ... ok

test result: ok. 8 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.18s
```

- 8つのテストは、同時に動いて、0.2 秒ほどで終わりました。どれも自分の DB を持っているので、同時に動いてもぶつかりません。

練習問題

問1: タイトル `Rust 入門`・本文 `SQLite の LIKE` のノートと、タイトル `Äpfel kaufen` のノートを作り、`Rust`・`rust`・`RUST`・`ｒｕｓｔ`（全角）・`sqlite`・`Äpfel`・`äpfel` で検索して、それぞれ何件見つかるかを表示してください。検索は、大文字と小文字を区別するでしょうか。

問2: セッションの期限を、決まった時刻 T にして作り、`find_user` を「T の1秒前」「T ちょうど」「T の1秒後」で呼んでください。`delete_expired` も「T の1秒前」と「T ちょうど」で呼び、何件消えたかを表示してください。期限ちょうどの時刻は、まだ使えるでしょうか。

問3: alice と bob に、ノートを2件ずつとセッションを1つずつ作ってから、SQL で alice を `users` 表から消し、`users`・`notes`・`sessions` の行の数が、消す前と後でどう変わるかを表示してください。

解答と解説

問1の解答:

`tests/u02_q1.rs` です。

```rust
//! 問1の解答: 検索（LIKE）は、大文字と小文字を区別するかを確かめる。
//! 実行: cargo test --test u02_q1 -- --nocapture

mod common;

use rustnote::{
    domain::{Email, NewNote, NoteQuery},
    repository::{LibsqlNoteRepository, LibsqlUserRepository, NoteRepository,
UserRepository},
};

#[tokio::test]
async fn like_and_letter_case() {
    let test = common::test_db().await;
    let users = LibsqlUserRepository::new(test.db.clone());
    let notes = LibsqlNoteRepository::new(test.db.clone());
    let alice = users
        .create(
            &Email::parse("alice@example.com").unwrap(),
            "$argon2id$dummy",
        )
        .await
        .unwrap();
    for (title, body) in [("Rust 入門", "SQLite の LIKE"), ("Äpfel kaufen", "")] {
        notes
            .insert(alice.id, &NewNote::parse(title, body).unwrap())
            .await
            .unwrap();
    }

    for keyword in [
        "Rust",
        "rust",
        "RUST",
        "ｒｕｓｔ",
        "sqlite",
        "Äpfel",
        "äpfel",
    ] {
        let page = notes
            .search(alice.id, &NoteQuery::new(Some(keyword), None))
            .await
            .unwrap();
        println!("{keyword:<8} → {} 件", page.total_count);
    }
}
```

```
cargo test --test u02_q1 -- --nocapture
```

```
running 1 test
Rust     → 1 件
rust     → 1 件
RUST     → 1 件
ｒｕｓｔ     → 0 件
sqlite   → 1 件
Äpfel    → 1 件
äpfel    → 0 件
test like_and_letter_case ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.04s
```

- `Rust`・`rust`・`RUST` は、どれも 1 件でした。SQLite の `LIKE` は、**英字（ASCII）の大文字と小文字を区別しません。**本文にだけある `sqlite` も見つかりました（タイトルと本文の両方を探す SQL だからです）。
- ところが、全角の `ｒｕｓｔ` は 0 件、`äpfel` も 0 件でした。SQLite の `LIKE` が大文字と小文字を同じに扱うのは、ASCII の英字だけで、`Ä` と `ä` のような文字は、別の文字として比べます。全角と半角も、別の文字です。
- これは、RustNote の仕様として決めておくことです。今の RustNote は「英字は大文字と小文字を区別しない。それ以外は、書いたとおりに探す」です。仕様を変えるなら、このテストの表を先に書き換えてから、SQL を直します。

問2の解答:

`tests/u02_q2.rs` です。

```rust
//! 問2の解答: セッションの期限ちょうどの時刻で、探す・掃除するとどうなるかを確かめる。
//! 実行: cargo test --test u02_q2 -- --nocapture

mod common;

use std::time::{Duration, SystemTime, UNIX_EPOCH};

use rustnote::{
    domain::Email,
    repository::{
        LibsqlSessionRepository, LibsqlUserRepository, SessionRepository,
UserRepository,
    },
};
use topcoat::session::TokenHash;

#[tokio::test]
async fn session_expiry_boundary() {
    let test = common::test_db().await;
    let users = LibsqlUserRepository::new(test.db.clone());
    let sessions = LibsqlSessionRepository::new(test.db.clone());
    let alice = users
        .create(
            &Email::parse("alice@example.com").unwrap(),
            "$argon2id$dummy",
        )
        .await
        .unwrap();

    // 期限（T）を、決まった時刻にしておく（今の時刻に頼らない）
    let t = UNIX_EPOCH + Duration::from_secs(2_000_000_000);
    let second = Duration::from_secs(1);
    let hash = TokenHash::new([7; 32]);
    sessions.create(&hash, alice.id, t).await.unwrap();

    for (label, now) in [
        ("T − 1秒", t - second),
        ("T ちょうど", t),
        ("T + 1秒", t + second),
    ] {
        let found = sessions.find_user(&hash, now).await.unwrap();
        println!("find_user（{label}）→ 見つかる: {}", found.is_some());
    }
    let before: SystemTime = t - second;
    println!(
        "delete_expired（T − 1秒）→ {} 件消した",
        sessions.delete_expired(before).await.unwrap()
    );
    println!(
```

```
        "delete_expired（T ちょうど）→ {} 件消した",
        sessions.delete_expired(t).await.unwrap()
    );
}
```

```
cargo test --test u02_q2 -- --nocapture
```

```
running 1 test
find_user（T − 1秒）→ 見つかる: true
find_user（T ちょうど）→ 見つかる: false
find_user（T + 1秒）→ 見つかる: false
delete_expired（T − 1秒）→ 0 件消した
delete_expired（T ちょうど）→ 1 件消した
test session_expiry_boundary ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.05s
```

- 期限ちょうどの T では、もう見つかりませんでした。`find_user` の SQL は `s.expires_at > ?2`（期限が今より**あと**なら有効）で、`delete_expired` は `expires_at <= ?1`（期限が今か、今より**前**なら消す）です。T ちょうどは「使えない」側と「消す」側の両方に入るので、すき間も重なりもありません。
- 境界の「ちょうど」は、UNIT 01 の境界値と同じで、いちばん間違えやすい所です。`>` と `>=` を取り違えると、「期限ちょうどの1秒だけ、使えないのに消されない」セッションができます。
- 時刻を `SystemTime::now()` ではなく、決まった時刻（`UNIX_EPOCH` から 20 億秒）にしたのは、テストの結果を、動かした時刻によって変えないためです（エラー2の「共有しているもの」の1つが、今の時刻です）。

問3の解答:

`tests/u02_q3.rs` です。

```rust
//! 問3の解答: ユーザーを消すと、その人のノートとセッションはどうなるかを確かめる。
//! 実行: cargo test --test u02_q3 -- --nocapture

mod common;

use std::time::{Duration, SystemTime};

use rustnote::{
    db::Db,
    domain::{Email, NewNote},
    repository::{
        LibsqlNoteRepository, LibsqlSessionRepository, LibsqlUserRepository,
NoteRepository,
        SessionRepository, UserRepository,
    },
};
use topcoat::session::TokenHash;

/// 表ごとの行の数を、1行の文字列にする
async fn counts(db: &Db) -> String {
    let conn = db.connect().await.expect("DB に接続できません");
    let mut parts = Vec::new();
    for table in ["users", "notes", "sessions"] {
        let mut rows = conn
            .query(&format!("SELECT COUNT(*) FROM {table}"), ())
            .await
            .expect("数えられません");
        let row = rows
            .next()
            .await
            .expect("行を読めません")
            .expect("行がない");
        let count: i64 = row.get(0).expect("COUNT(*)");
        parts.push(format!("{table}={count}"));
    }
    parts.join(" ")
}

#[tokio::test]
async fn deleting_a_user_cascades() {
    let test = common::test_db().await;
    let users = LibsqlUserRepository::new(test.db.clone());
    let notes = LibsqlNoteRepository::new(test.db.clone());
    let sessions = LibsqlSessionRepository::new(test.db.clone());
    let expires = SystemTime::now() + Duration::from_secs(60);
    for (n, email) in ["alice@example.com", "bob@example.com"]
        .into_iter()
        .enumerate()
    {
```

```
        let user = users
            .create(&Email::parse(email).unwrap(), "$argon2id$dummy")
            .await
            .unwrap();
        for title in ["1件目", "2件目"] {
            notes
                .insert(user.id, &NewNote::parse(title, "").unwrap())
                .await
                .unwrap();
        }
        let hash = TokenHash::new([n as u8; 32]);
        sessions.create(&hash, user.id, expires).await.unwrap();
    }
    println!("消す前: {}", counts(&test.db).await);

    // RustNote には「退会」の機能がまだ無いので、SQL で直接 alice を消す
    let conn = test.db.connect().await.unwrap();
    let deleted = conn
        .execute("DELETE FROM users WHERE email = 'alice@example.com'", ())
        .await
        .unwrap();
    println!("users から {deleted} 行消した");
    println!("消した後: {}", counts(&test.db).await);
}
```

```
cargo test --test u02_q3 -- --nocapture
```

```
running 1 test
消す前: users=2 notes=4 sessions=2
users から 1 行消した
消した後: users=1 notes=2 sessions=1
test deleting_a_user_cascades ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.07s
```

- `users` から alice の1行を消すと、alice のノート2件とセッション1つも、一緒に消えました。bob の分は残っています。
- `migrations` の表の定義で、`notes.user_id` と `sessions.user_id` に `REFERENCES users(id) ON DELETE CASCADE`（参照先が消えたら、一緒に消す）と書いてあり、libSQL の接続では外部キーの検査が有効（`foreign_keys=1`。第13巻 UNIT 02）だからです。
- もし外部キーの検査が無効だったら、持ち主のいないノートが残ります。こうした**表の約束**も、SQL と同じで、本物の DB でしか確かめられません。RustNote に「退会」の機能を作るときは、このテストを、その機能のテストに育てます。

まとめ

- リポジトリは、本物の DB（一時フォルダの libSQL ファイル）で確かめる。Rust の関数が正しくても、SQL の `ESCAPE` や `WHERE` の持ち主の条件が壊れていれば、ユニットテストは通ったままになる。
- 準備・操作・確かめの形で、成功だけでなく、見つからない・他人・版の食い違いを1つずつ確かめる。`assert!` には、失敗したときの文と実際の値を書いておく。
- テストごとに新しい DB を作り、テストどうしで何も共有しない。共有すると、結果が実行のたびに変わり、前の実行のデータにも引きずられる。

UNIT 03: HTTP のテスト

この単元のゴール

- レスポンスの4つの部分（状態コード・ヘッダー・クッキー・本文）を、RustNote の本物のレスポンスで読み、それぞれ何を確かめればよいかを言える
- 確かめの道具（`assert_redirect`・`cookie_attributes`）を作り、`#[track_caller]` を付けて、失敗したときに**テストのどの行か**が出るようにできる
- 本文を「その文字があるか」（`contains`）ではなく、CSS セレクター（`scraper`）で「どの要素の中か」まで決めて確かめる理由を、通ってしまう間違いで説明できる
- 同じ名前のヘッダー（`Set-Cookie`）が2つあるときの読み方と、順番に頼ったテストが、ときどきだけ失敗する理由を言える

たとえ話でつかむ

通販の荷物を受け取って、中身を確かめる（検品する）場面を考えます。

- **伝票の印**: 「お届け済み」「転送しました」のような印が、まず目に入ります。これが**状態コード**です。「転送しました」なら、**転送先の住所**（`Location`）も確かめます。
- **同封の会員証**: 箱に、会員証が入っていることがあります。会員証には「有効期限」「本人だけが使えます」のような**注意書き**（属性）が付いています。会員証が2枚入っているとき、どちらが上に入っているかは、**決まっていません**。だから、上から1枚目ではなく、**名前で**探します。
- **箱の中身**: 「箱のどこかに『リンゴ』という字がある」では、検品になりません。中に入っていたチラシに「リンゴ」と書いてあるだけかもしれないからです。「**果物の仕切りの中に**、リンゴが1個」と、**場所まで決めて**確かめます。

荷物の検品	HTTP のテスト
伝票の印	状態コード（`200 OK`・`303 See Other`・`404 Not Found` など）
転送先の住所	`Location` ヘッダー
会員証と注意書き	`Set-Cookie` ヘッダーと、その属性（`HttpOnly`・`Max-Age` など）
会員証が2枚	同じ名前のヘッダーが2つ（`get_all` で全部読み、名前で探す）
箱の中身	本文（HTML）
仕切りの中で確かめる	CSS セレクターで、要素を決めて確かめる（例3-3）

> **用語:** ヘッダー（レスポンスの本文の前に付く「名前: 値」の行。移動先の `Location`、クッキーを渡す `Set-Cookie`、本文の形を表す `Content-Type` など。同じ名前の行が、いくつあってもよい）

> **用語:** CSS セレクター（HTML の中の要素を、`ul.note-list li a`（`note-list` という class の `ul` の中の、`li` の中の `a`）のように、場所で指定する書き方。CSS で見た目を当てるときと同じ書き方）

> **用語:** `#[track_caller]`（関数に付ける印。その関数の中で失敗（panic）したとき、失敗の場所として、その関数を**呼んだ行**が表示されるようになる）

図で理解する

RustNote の `POST /signup` のレスポンスの4つの部分と、それぞれの確かめ方です（例3-1の出力から）。

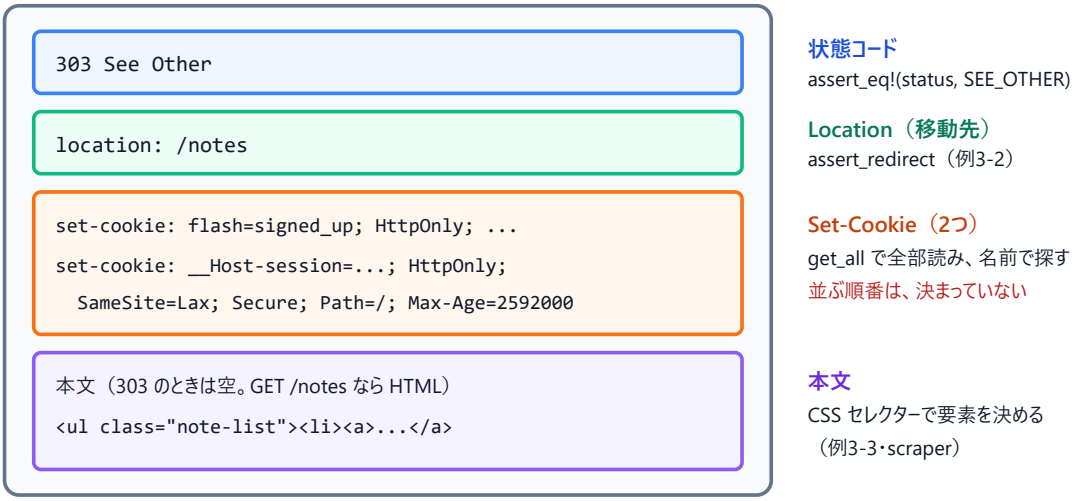


図11-8: レスポンスの4つの部分。状態コードと Location は値で、Set-Cookie は名前で探して属性まで、本文は要素を決めて確かめる

検索の画面で、「買う」という字が、どこにあるかです。

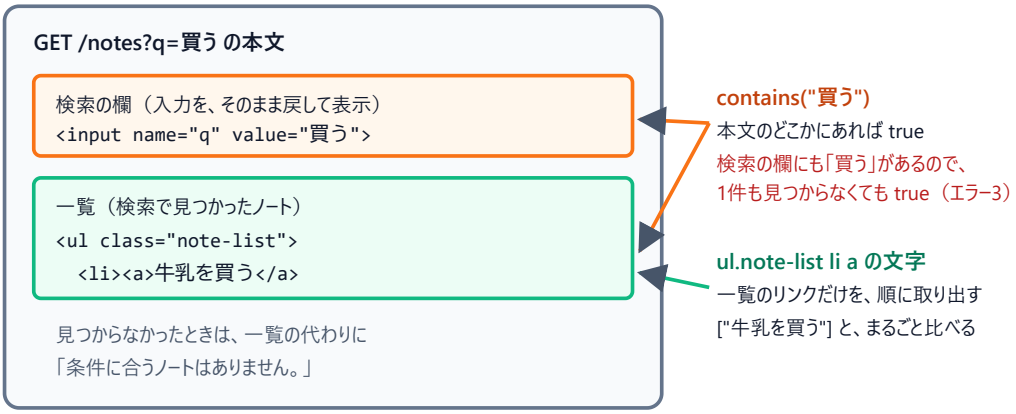


図11-9: 検索の画面の「買う」。contains は検索の欄と一覧の両方を見てしまうので、一覧が空でも通る。CSS セレクターなら、一覧のリンクだけを見る

まず動かす最小コード

例3-1: 1つのレスポンスの、4つの部分を見る

テストを書く前に、RustNote が**本当に何を返しているか**を、全部表示して見ます。

> **▼ このコードがやること（先に日本語で）**：登録のフォームを送り（ `POST /signup` ）、返ってきた状態コードと、ヘッダーを全部表示します。セッションのトークンは、先頭の6文字と長さだけにします。次に、そのクッキーでノートの一覧（ `GET /notes` ）を開き、状態コード・ヘッダーと、本文の長さ・ `<title>` ・ `<h1>` を表示します。

```rust
//! 第11巻 UNIT 03: 1つのレスポンスの4つの部分（状態コード・ヘッダー・クッキー・本文）を見る。
//! 実行: cargo test --test u03_parts -- --nocapture

mod common;

use common::{body_text, encode, session_cookie, test_app};
use topcoat::router::response::Response;

const PASSWORD: &str = "correct horse battery";

/// 状態コードと、ヘッダーを全部表示する（セッションのトークンは、先頭の6文字だけ）
fn show(label: &str, response: &Response) {
    println!("--- {label}");
    println!("状態コード: {}", response.status());
    for (name, value) in response.headers() {
        let value = value.to_str().unwrap_or("（文字にできない）");
        let shown = match value.strip_prefix("__Host-session=") {
            Some(rest) => {
                let (token, attributes) = rest.split_once(';').unwrap_or((rest, ""));
                format!(
                    "__Host-session={}…（{} 文字）;{attributes}",
                    &token[..6],
                    token.len()
                )
            }
            None => value.to_string(),
        };
        println!("{name}: {shown}");
    }
}

#[tokio::test]
async fn four_parts_of_a_response() {
    let app = test_app().await;

    // 1. 登録のフォームを送る（成功すると、303 でノートの一覧へ）
    let form = format!(
        "email={}&password={}&password_confirm={}",
        encode("alice@example.com"),
        encode(PASSWORD),
        encode(PASSWORD)
    );
    let response = app.post_form("/signup", &form, None).await;
    show("POST /signup", &response);
    let cookie = session_cookie(&response).unwrap();

    // 2. ログインしたクッキーで、ノートの一覧を開く
    let response = app.get("/notes", Some(&cookie)).await;
    show("GET /notes", &response);
```

```
    let html = body_text(response).await;
    println!("本文: {} バイト", html.len());
    for tag in ["<title>", "<h1>"] {
        let start = html.find(tag).unwrap();
        let end = start + html[start..].find("</").unwrap();
        println!("本文の {tag}: {}", &html[start + tag.len()..end]);
    }
}
```

コピーした `rustnote` フォルダの `tests/u03_parts.rs` として保存して実行します。

```
cargo test --test u03_parts -- --nocapture
```

▼ 実行結果

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.83s
     Running tests\u03_parts.rs (target\debug\deps\u03_parts-76b358a1ad142f58.exe)

running 1 test
POST /signup -> 303 See Other (511.5942ms)
--- POST /signup
状態コード: 303 See Other
location: /notes
set-cookie: flash=signed_up; HttpOnly; SameSite=Lax; Path=/; Max-Age=300
set-cookie: __Host-session=Vr9LDo…（46 文字）; HttpOnly; SameSite=Lax; Secure; Path=/;
Max-Age=2592000
GET /notes -> 200 OK (9.328ms)
--- GET /notes
状態コード: 200 OK
content-type: text/html; charset=utf-8
vary: Accept
vary: accept-encoding
本文: 1318 バイト
本文の <title>: RustNote
本文の <h1>: ノート
test four_parts_of_a_response ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.55s
```

▼ コードを1つずつ分解して解説

解説1: 状態コードと Location

- 登録に成功すると、`303 See Other` と `location: /notes` が返りました。ブラウザは、この2つを見て、`/notes` を開き直します（PRG。第10巻 UNIT 02）。本文は空です。

- `303` だけを確かめても、移動先が違えば、利用者は別の画面に行ってしまいます。**303 と Location は、いつも組で**確かめます（例3-2の `assert_redirect`）。

解説2: Set-Cookie は2つ。属性まで読む

- `set-cookie:` の行が2つありました。お知らせの `flash=signed_up` と、ログインの `__Host-session` です。同じ名前のヘッダーが、2回出てきます。
- `__Host-session` の属性は、`HttpOnly`（JavaScript から読めない）・`SameSite=Lax`（ほかのサイトからの POST では送られない）・`Secure`（HTTPS だけ）・`Path=/`・`Max-Age=2592000`（2,592,000 秒＝30 日）です。`flash` は `Max-Age=300`（5 分）で、`Secure` がありません。どれも第8巻で確かめた守りなので、テストで、属性まで確かめます。
- この出力では `flash` が先ですが、**先とは限りません**（エラー2）。

解説3: GET /notes のヘッダーと本文

- `content-type: text/html; charset=utf-8` は、「本文は HTML で、文字は UTF-8」という意味です。RustNote は、HTML の中にも `<meta charset="utf-8">` を書いていますが、ブラウザは、ヘッダーのほうを先に見ます。
- `vary:` は、Topcoat が付けた、「`Accept`・`accept-encoding` のヘッダーによって、返す中身が変わることがある」というキャッシュ向けの知らせです。
- 本文は 1,318 バイトで、`<title>` は `RustNote`、`<h1>` は `ノート` でした。
- 逆に、**無いヘッダー**もあります。`X-Frame-Options` のような、守りのためのヘッダーは1つもありませんでした（問2）。全部表示してみると、こうした「無いもの」にも気づけます。

もう一歩: 実務に近い形

例3-2: 確かめの道具を作り、`#[track_caller]` を付ける

例3-1で見た値を、`assert` で確かめます。「303 で、この URL へ」「このクッキーの属性は、これ」は何度も出てくるので、**道具**（小さな関数）にします。

> **▼ このコードがやること（先に日本語で）：** `assert_redirect` は、「303 で、`to` へ移動するか」を確かめます。`cookie_attributes` は、`Set-Cookie` を**全部**読んで、名前が合うものを探し、属性を並んだ順に返します。この2つの道具で、登録のレスポンス・ログインしていない人の移動先・Content-Type を確かめる、3つのテストを書きます。

```rust
//! 第11巻 UNIT 03: レスポンスを確かめる道具（ヘルパー）を作り、HTTP のテストを短く書く。
//! 道具に #[track_caller] を付けて、失敗したときに「テストのどの行か」が出るようにする。
//!
//! 実行: cargo test --test u03_helpers

mod common;

use common::{encode, location, test_app};
use topcoat::router::{StatusCode, header, response::Response};

const PASSWORD: &str = "correct horse battery";

/// 303 で `to` へ移動するレスポンスか
#[track_caller]
fn assert_redirect(response: &Response, to: &str) {
    assert_eq!(response.status(), StatusCode::SEE_OTHER, "303 のはず");
    assert_eq!(location(response), to, "移動先");
}

/// `name` のクッキーの属性（`HttpOnly`・`Path=/` など）を、並んだ順に返す。無ければ止まる
#[track_caller]
fn cookie_attributes(response: &Response, name: &str) -> Vec<String> {
    let prefix = format!("{name}=");
    let value = response
        .headers()
        .get_all(header::SET_COOKIE)
        .iter()
        .filter_map(|value| value.to_str().ok())
        .find(|value| value.starts_with(&prefix))
        .unwrap_or_else(|| panic!("{name} のクッキーがありません"));
    value
        .split(';')
        .skip(1)
        .map(|attribute| attribute.trim().to_string())
        .collect()
}

#[tokio::test]
async fn signup_sets_two_cookies_and_redirects() {
    let app = test_app().await;
    let form = format!(
        "email={}&password={}&password_confirm={}",
        encode("alice@example.com"),
        encode(PASSWORD),
        encode(PASSWORD)
    );
    let response = app.post_form("/signup", &form, None).await;

    assert_redirect(&response, "/notes");
```

```
    // セッション: JavaScript から読めず・HTTPS だけ・ほかのサイトからの POST では送られず・30
日
    assert_eq!(
        cookie_attributes(&response, "__Host-session"),
        [
            "HttpOnly",
            "SameSite=Lax",
            "Secure",
            "Path=/",
            "Max-Age=2592000"
        ]
    );
    // お知らせ: 5 分だけ
    assert_eq!(
        cookie_attributes(&response, "flash"),
        ["HttpOnly", "SameSite=Lax", "Path=/", "Max-Age=300"]
    );
}

#[tokio::test]
async fn pages_for_members_send_guests_to_login() {
    let app = test_app().await;
    for path in ["/notes", "/notes/1", "/notes/1/edit"] {
        let response = app.get(path, None).await;
        assert_redirect(&response, "/login");
    }
}

#[tokio::test]
async fn html_pages_say_utf8() {
    let app = test_app().await;
    let response = app.get("/login", None).await;
    assert_eq!(response.status(), StatusCode::OK);
    assert_eq!(
        response.headers()[header::CONTENT_TYPE],
        "text/html; charset=utf-8"
    );
}
```

```
cargo test --test u03_helpers
```

▼ 実行結果

```
running 3 tests
test html_pages_say_utf8 ... ok
test pages_for_members_send_guests_to_login ... ok
test signup_sets_two_cookies_and_redirects ... ok
test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.49s
```

- 3つとも通りました。`cookie_attributes(&response, "__Host-session")` と書けば、何を確かめているかが、読む人にもすぐ分かります。
- 道具の頭に付けた `#[track_caller]` は、**失敗したときのため**の印です。違いを見るために、`/notes/1` の移動先を、わざと `/notes` と書きまちがえたテストを、印の無い版と有る版で動かします（`assert_redirect` を3回呼び、2回目だけが間違い）。

```
//! 第11巻 UNIT 03 例3-2: 道具に #[track_caller] が無いと、失敗の場所が道具の中になる。
//! 3つの URL のうち、どれか1つの期待を、わざと書きまちがえている。
//! 実行: cargo test --test e03_no_track

mod common;

use common::{location, test_app};
use topcoat::router::{StatusCode, response::Response};

fn assert_redirect(response: &Response, to: &str) {
    assert_eq!(response.status(), StatusCode::SEE_OTHER, "303 のはず");
    assert_eq!(location(response), to, "移動先");
}

#[tokio::test]
async fn guests_are_redirected() {
    let app = test_app().await;
    assert_redirect(&app.get("/notes", None).await, "/login");
    assert_redirect(&app.get("/notes/1", None).await, "/notes");
    assert_redirect(&app.get("/notes/1/edit", None).await, "/login");
}
```

印が無い版（`tests/e03_no_track.rs`）：

```
thread 'guests_are_redirected' (218940) panicked at tests\e03_no_track.rs:12:5:
assertion `left == right` failed: 移動先
  left: "/login"
 right: "/notes"
```

印が有る版（`tests/e03_track.rs`。違うのは `#[track_caller]` の1行だけ）：

```
thread 'guests_are_redirected' (217604) panicked at tests\e03_track.rs:20:5:
assertion `left == right` failed: 移動先
  left: "/login"
 right: "/notes"
```

- 印が無いと、失敗の場所は `e03_no_track.rs:12:5`、つまり**道具の中**の `assert_eq!` の行です。3回呼んだうちの、どれが失敗したのかが分かりません。
- 印が有ると、`e03_track.rs:20:5`、つまり**2回目に呼んだ行**が出ます。道具を作ったら、`#[track_caller]` を付けるのを、くせにします。

例3-3: 本文を、CSS セレクターで確かめる

本文の HTML を、文字の並びのまま調べるのではなく、**HTML として読んで**、要素を決めて調べます。HTML を読むクレート `scraper` を、練習用のコピーの dev-dependency（テストのときだけ使う依存）に足します。

```
cargo add --dev scraper@=0.29.0
```

```
    Updating crates.io index
      Adding scraper =0.29.0 to dev-dependencies
             Features:
             + errors
             + main
             - atomic
             - deterministic
             - serde
    Updating crates.io index
     Locking 28 packages to highest Rust 1.99 compatible versions
```

（このあと、`scraper` 自身と、`scraper` が使うクレートの、合わせて 28 個の `Adding` の行が続きます。）

> **▼ このコードがやること（先に日本語で）**：alice で「牛乳を買う」「本を返す」の2つのノートを作り、「買う」で検索した一覧の画面を、2つのやり方で確かめます。1つ目は、本文に「買う」という字があるか（ `contains` ）。2つ目は、`ul.note-list li a`（一覧のリンク）の文字を、上から順に取り出し、`["牛乳を買う"]` とまるごと比べます。

```rust
//! 第11巻 UNIT 03: 本文の確かめ方を比べる。
//! 「本文に、その文字があるか」（contains）と、「一覧のリンクに、その題があるか」（scraper の CSS セレクター）。
//!
//! 実行: cargo test --test u03_body -- --nocapture

mod common;

use common::{body_text, test_app};
use scraper::{Html, Selector};

const PASSWORD: &str = "correct horse battery";

/// 一覧のページの、ノートのリンクの文字を、上から順に返す
fn listed_titles(html: &str) -> Vec<String> {
    let document = Html::parse_document(html);
    let links = Selector::parse("ul.note-list li a").unwrap();
    document
        .select(&links)
        .map(|link| link.text().collect::<String>())
        .collect()
}

#[tokio::test]
async fn search_by_contains() {
    let app = test_app().await;
    let alice = app.sign_up("alice@example.com", PASSWORD).await;
    app.create_note(&alice, "牛乳を買う", "").await;
    app.create_note(&alice, "本を返す", "").await;

    let html = body_text(app.get("/notes?q=%E8%B2%B7%E3%81%86",
Some(&alice)).await).await;
    println!("contains(\"買う\") → {}", html.contains("買う"));
    assert!(
        html.contains("買う"),
        "「買う」で検索すると、本文に「買う」がある"
    );
}

#[tokio::test]
async fn search_by_selector() {
    let app = test_app().await;
    let alice = app.sign_up("alice@example.com", PASSWORD).await;
    app.create_note(&alice, "牛乳を買う", "").await;
    app.create_note(&alice, "本を返す", "").await;

    let html = body_text(app.get("/notes?q=%E8%B2%B7%E3%81%86",
Some(&alice)).await).await;
    let titles = listed_titles(&html);
```

```
    println!("一覧のリンク → {titles:?}");
    assert_eq!(
        titles,
        ["牛乳を買う"],
        "「買う」で検索すると、一覧は「牛乳を買う」だけ"
    );
}
```

```
cargo test --test u03_body -- --nocapture --test-threads=1
```

▼ 実行結果

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 1.57s
     Running tests\u03_body.rs (target\debug\deps\u03_body-3a77394819c142a3.exe)

running 2 tests
test search_by_contains ... POST /signup -> 303 See Other (425.0911ms)
POST /notes -> 303 See Other (7.3585ms)
POST /notes -> 303 See Other (9.0424ms)
GET /notes -> 200 OK (5.4579ms)
contains("買う") → true
ok
test search_by_selector ... POST /signup -> 303 See Other (410.4965ms)
POST /notes -> 303 See Other (6.984ms)
POST /notes -> 303 See Other (8.6498ms)
GET /notes -> 200 OK (3.9642ms)
一覧のリンク → ["牛乳を買う"]
ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.91s
```

- どちらも通りました。ここだけ見ると、同じに見えます。でも、確かめている中身は違います（図の2つ目）。違いは、検索がまちがっていたときに出ます（エラー3）。
- `--test-threads=1` は、2つのテストを1つずつ動かす指定です。同時に動かすと、アクセスログの行と `println!` の行が、行の途中で混ざって読みにくくなるので、ここでは1つずつにしました。
- `link.text()` は、要素の中の文字を返します。HTML の `&` のような書き方は、もとの文字（`&`）に戻ります（問3）。

よくあるエラーと読み方

エラー1: 本文を読んだあとで、状態コードを見ようとした（E0382）

```
//! 第11巻 UNIT 03 エラー1: 本文を読んだあとで、同じレスポンスの状態コードを見ようとした。
//! 実行: cargo test --test e03_moved

mod common;

use common::{body_text, test_app};
use topcoat::router::StatusCode;

#[tokio::test]
async fn login_page_has_a_form() {
    let app = test_app().await;
    let response = app.get("/login", None).await;

    let html = body_text(response).await;
    assert!(html.contains("<form"));
    assert_eq!(response.status(), StatusCode::OK);
}
```

```
cargo test --test e03_moved
```

```
   Compiling rustnote v0.1.0 (...\rustnote)
error[E0382]: borrow of moved value: `response`
   --> tests\e03_moved.rs:16:16
    |
 12 |     let response = app.get("/login", None).await;
    |         -------- move occurs because `response` has type
`http::response::Response<Body>`, which does not implement the `Copy` trait
 13 |
 14 |     let html = body_text(response).await;
    |                          -------- value moved here
 15 |     assert!(html.contains("<form"));
 16 |     assert_eq!(response.status(), StatusCode::OK);
    |                ^^^^^^^^ value borrowed here after move
    |
note: consider changing this parameter type in function `body_text` to borrow instead
if owning the value isn't necessary
   --> tests\common\mod.rs:117:34
    |
117 | pub async fn body_text(response: Response) -> String {
    |              ---------           ^^^^^^^^ this parameter takes ownership of the
value
    |              |
    |              in this function

For more information about this error, try `rustc --explain E0382`.
error: could not compile `rustnote` (test "e03_moved") due to 1 previous error
```

- **何が起きたか:**「`response` は、14 行目で `body_text` に**移された**（move）ので、16 行目では、もう使えません」というエラーです。`body_text(response: Response)` は、レスポンスを引数として**受け取って**（所有権ごと）本文を読みます（第3巻）。
- **直し方:** 本文を読む**前に**、状態コードやヘッダーを、変数に取っておきます。

```
    let status = response.status();
    let html = body_text(response).await;
    assert_eq!(status, StatusCode::OK);
```

問1の答えの `tests/u03_q1.rs` も、この順番で書いています。

エラー2（ときどき失敗する）：1つ目の Set-Cookie を、セッションのクッキーだと思い込んだ

```rust
//! 第11巻 UNIT 03 エラー2: 1つ目の Set-Cookie が、セッションのクッキーだと思い込んだテスト。
//! 実行: cargo test --test e03_first_cookie

mod common;

use common::{encode, test_app};
use topcoat::router::header;

const PASSWORD: &str = "correct horse battery";

#[tokio::test]
async fn first_set_cookie_is_the_session() {
    let app = test_app().await;
    let form = format!(
        "email={}&password={}&password_confirm={}",
        encode("alice@example.com"),
        encode(PASSWORD),
        encode(PASSWORD)
    );
    let response = app.post_form("/signup", &form, None).await;

    // get は、同じ名前のヘッダーが2つあっても、1つ目しか返さない
    let first = response.headers()[header::SET_COOKIE].to_str().unwrap();
    assert!(
        first.starts_with("__Host-session="),
        "1つ目の Set-Cookie: {first}"
    );
}
```

このテストを、10 回続けて動かしました。

```
1 回目: ok
2 回目: ok
3 回目: ok
4 回目: ok
5 回目: ok
6 回目: ok
7 回目: ok
8 回目: ok
9 回目: ok
10 回目: FAILED
```

失敗した回の出力です。

```
test first_set_cookie_is_the_session ... FAILED
thread 'first_set_cookie_is_the_session' (213976) panicked at
tests\e03_first_cookie.rs:24:5:
1つ目の Set-Cookie: flash=signed_up; HttpOnly; SameSite=Lax; Path=/; Max-Age=300
test result: FAILED. 0 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.46s
```

- **何が起きたか**: 10 回のうち、9 回は通り、1 回だけ失敗しました。失敗した回は、1つ目の `Set-Cookie` が `flash=signed_up` でした。もう一度 10 回試したときは、6 回失敗しました。RustNote は、1回も変わっていません。
- **なぜか**: `headers()[…]`（と `headers().get(…)`）は、同じ名前のヘッダーが2つあっても、**1つ目だけ**を返します。そして、Topcoat が使っている cookie 0.18.2 は、返すクッキーを `HashSet`（順番を持たない入れ物）にしまっています。`HashSet` の中の順番は、プログラムを動かすたびに変わるので、どちらが1つ目になるかは、その時しだいです。
- **直し方**: 例3-2の `cookie_attributes` のように、`get_all` で**全部**読んで、**名前で**探します。RustNote の `tests/common/mod.rs` の `session_cookie` も、この形です。
- **学べること**:「ときどきだけ失敗するテスト」は、順番の決まっていないもの（`HashSet`・`HashMap` の並び・同時に動くテストの順番・時刻）に頼っていることがよくあります（UNIT 02 のエラー2も、同時に動くテストの順番でした）。

エラー3（通ってしまう間違い）：検索を「前に一致」にしてしまった

検索の模様を作る行で、`format!("%{}%", …)`（どこかに含む）を、`format!("{}%", …)`（その字で**始まる**）と書きまちがえた場合です（練習用のコピーの `src/repository/notes.rs` で試し、必ず元に戻します）。

```
            Some(keyword) => format!("{}%", escape_like(keyword)),
```

例3-3のテストを動かします。

```
test search_by_contains ... POST /signup -> 303 See Other (423.5922ms)
contains("買う") → true
test search_by_selector ... POST /signup -> 303 See Other (440.359ms)
一覧のリンク → []
thread 'search_by_selector' (198124) panicked at tests\u03_body.rs:48:5:
assertion `left == right` failed: 「買う」で検索すると、一覧は「牛乳を買う」だけ
  left: []
 right: ["牛乳を買う"]
test result: FAILED. 1 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.99s
```

- **何が起きたか**:「牛乳を買う」は「買う」で**始まらない**ので、一覧は空（`[]`）になりました。それなのに、`contains("買う")` の版は `true` のまま**通りました**。検索の欄に、入力した「買う」が、そのまま戻して表示されているからです。セレクターの版だけが、「一覧は `[]`」と止まりました。

- **RustNote のテストでは:** 同じ書きまちがいのまま、RustNote の `tests/http.rs` と `tests/repository.rs` も動かしました。

```
test result: ok. 14 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished
in 2.01s
test search_filters_by_keyword_and_paginates ... FAILED
thread 'search_filters_by_keyword_and_paginates' (211964) panicked at
tests\repository.rs:203:5:
test result: FAILED. 7 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.19s
```

- `tests/http.rs` の 14 個は、**全部通りました**。検索のテスト（`list_searches_and_paginates`）は「所有権」で探しますが、そのノートの本文は「所有権」そのもので、「所有権」で**始まる**ので、前に一致でも見つかるからです。`tests/repository.rs` の検索のテストが、203 行目（`_計算` で探す所）で気づきました。
- **学べること:** 本文の確かめは、「どの要素の中に、何が、いくつあるか」まで決めます。そして、テストに使うデータは、間違いが表に出る形（キーワードが**真ん中**にある題）を選びます。HTTP のテストと、リポジトリの結合テストは、おたがいに見落としを拾い合います。

RustNote ではここで使う

- **HTTP のテストの形:** RustNote の `tests/http.rs` は、ポートを開かずに、`Router::handle` へリクエストを直接渡して確かめます（`tests/common/mod.rs` の `test_app`・`get`・`post_form`）。この単元のテストも、同じ道具を使いました。
- **Set-Cookie:** `common::session_cookie` は、`get_all` で全部読み、`__Host-session=` で始まるものを探します（エラー2の直し方の形）。一方、`notes_require_login` は、`get(header::SET_COOKIE)` で1つ目を読んでいます。ログインしていない人へのレスポンスには、クッキーが `flash` の1つしかないので、今は正しく動きますが、クッキーが増えると、エラー2と同じことが起きます（改善点）。
- **本文:** `tests/http.rs` の本文の確かめは、ほとんどが `contains` です。一覧の件数は `html.matches("<li class=").count()` で数えています。エラー3のように、`contains` が通ってしまう所が無いかを、`scraper` で見直す余地があります（改善点。RustNote の `Cargo.toml` には、`scraper` を足していません）。
- **守りのヘッダー:** RustNote は、`X-Frame-Options` などの守りのヘッダーを返していません（問2、改善点）。

練習問題

問1: ログインしていない人が `GET /notes` を開いたときの、状態コード・`Location`・`Set-Cookie`・本文を、例3-1のように表示し、`assert` で確かめてください。移動先のログイン画面に「ログインしてください」が出ることも確かめてください。

問2: `GET /login` と、ログインした人の `GET /notes` のレスポンスに、次の6つのヘッダーがあるかを表示してください。`x-content-type-options`・`x-frame-options`・`content-security-policy`・`referrer-policy`・`strict-transport-security`・`cache-control`。無いと、どんなことが起こりうるでしょうか。

問3: 題が `<b>太字</b> & "引用"` のノートを作り、一覧の画面で、`ul.note-list li a` の文字が、入力した題と同じであることと、一覧の中に `b` 要素が1つも無いことを、`scraper` で確かめてください。本文の中では、どんな形で書かれているでしょうか。

解答と解説

問1の解答:

```rust
//! 第11巻 UNIT 03 問1: ログインしていない人が GET /notes を開いたときの、4つの部分を確かめる。
//! 実行: cargo test --test u03_q1 -- --nocapture

mod common;

use common::{body_text, location, test_app};
use topcoat::router::{StatusCode, header};

#[tokio::test]
async fn guest_opens_notes() {
    let app = test_app().await;
    let response = app.get("/notes", None).await;

    let status = response.status();
    let to = location(&response).to_string();
    let cookies: Vec<String> = response
        .headers()
        .get_all(header::SET_COOKIE)
        .iter()
        .map(|value| value.to_str().unwrap().to_string())
        .collect();
    let html = body_text(response).await;
    println!("状態コード: {status}");
    println!("Location: {to}");
    println!("Set-Cookie: {cookies:?}");
    println!("本文: {} バイト", html.len());

    assert_eq!(status, StatusCode::SEE_OTHER);
    assert_eq!(to, "/login");
    assert_eq!(
        cookies,
        ["flash=login_required; HttpOnly; SameSite=Lax; Path=/; Max-Age=300"]
    );

    // 移動先のログイン画面で、お知らせが出る
    let html = body_text(app.get("/login", Some("flash=login_required")).await).await;
    println!(
        "移動先に「ログインしてください」: {}",
        html.contains("ログインしてください")
    );
}
```

```
cargo test --test u03_q1 -- --nocapture
```

```
状態コード: 303 See Other
Location: /login
Set-Cookie: ["flash=login_required; HttpOnly; SameSite=Lax; Path=/; Max-Age=300"]
本文: 0 バイト
移動先に「ログインしてください」: true
```

- `303`・`/login`・`flash=login_required`（5 分）の3つが、ログインの守り（`require_user`）の「出口」です。本文は 0 バイトでした。
- エラー1のとおり、`status`・`to`・`cookies` を、`body_text` の**前に**取り出しています。
- 移動先の確かめでは、`Set-Cookie` の値の `flash=login_required` を、そのまま `Cookie` として送りました。ブラウザが次のリクエストでするのと同じです。

問2の解答:

```rust
//! 第11巻 UNIT 03 問2: 守りに関わるヘッダーが、レスポンスにあるかを確かめる。
//! 実行: cargo test --test u03_q2 -- --nocapture

mod common;

use common::test_app;

const NAMES: [&str; 6] = [
    "x-content-type-options",
    "x-frame-options",
    "content-security-policy",
    "referrer-policy",
    "strict-transport-security",
    "cache-control",
];

#[tokio::test]
async fn security_headers() {
    let app = test_app().await;
    let alice = app
        .sign_up("alice@example.com", "correct horse battery")
        .await;
    for (path, cookie) in [("/login", None), ("/notes", Some(alice.as_str()))] {
        let response = app.get(path, cookie).await;
        println!("--- GET {path} ({}) ", response.status());
        for name in NAMES {
            let value = response.headers().get(name);
            println!(
                "{name:<26} {}",
                value.map_or("無い".to_string(), |v| v.to_str().unwrap().to_string())
            );
        }
    }
}
```

```
--- GET /login (200 OK)
x-content-type-options     無い
x-frame-options            無い
content-security-policy    無い
referrer-policy            無い
strict-transport-security  無い
cache-control              無い
--- GET /notes (200 OK)
x-content-type-options     無い
x-frame-options            無い
content-security-policy    無い
referrer-policy            無い
strict-transport-security  無い
cache-control              無い
```

- 6つとも、どちらのページにもありませんでした。それぞれ、次のことを防ぐためのヘッダーです。

ヘッダー	無いと、起こりうること
`x-content-type-options: nosniff`	ブラウザが、`Content-Type` と違う種類のファイルだと推し量って、扱ってしまう
`x-frame-options`（または CSP の `frame-ancestors`）	ほかのサイトが、RustNote の画面を `iframe` の中に見えないように重ね、利用者に、知らずにボタンを押させる（クリックジャッキング）
`content-security-policy`	もし HTML に、ほかの人のスクリプトが紛れ込んでも（XSS）、ブラウザが止めない
`referrer-policy`	ほかのサイトへのリンクを押したとき、元の URL をどこまで相手に伝えるかが、ブラウザの既定まかせになる（今の多くのブラウザは、ほかのサイトにはドメインまでしか伝えないが、古いブラウザでは `/notes?q=…` の検索の言葉まで伝わる）
`strict-transport-security`	利用者が `http://` で開くたびに、HTTPS へ移る前の最初のリクエストが、暗号化されないまま送られる（途中で書き換えられるおそれ）
`cache-control: no-store`	ログインした人だけの画面が、ブラウザのキャッシュや履歴（戻るボタン）に残ることがある

- RustNote は、第8巻で、クッキーの `SameSite=Lax`（ほかのサイトの `iframe` の中では、ログインのクッキーが送られない）と、Topcoat のエスケープ（問3）で、いちばん大きな危険を防いでいます。それでも、何重にも守るために、これらのヘッダーを足す余地があります（改善点）。どの値にするかは、UNIT 06（準備中）の守りの回帰テストで、もう一度考えます。

問3の解答:

```rust
//! 第11巻 UNIT 03 問3: 題に HTML の記号を入れたノートが、一覧で「ただの文字」として出るかを、
//! CSS セレクターで確かめる。
//! 実行: cargo test --test u03_q3 -- --nocapture

mod common;

use common::{body_text, test_app};
use scraper::{Html, Selector};

#[tokio::test]
async fn titles_are_text_not_html() {
    let app = test_app().await;
    let alice = app
        .sign_up("alice@example.com", "correct horse battery")
        .await;
    let title = r#"<b>太字</b> & "引用""#;
    app.create_note(&alice, title, "").await;

    let html = body_text(app.get("/notes", Some(&alice)).await).await;
    let document = Html::parse_document(&html);
    let links: Vec<String> = document
        .select(&Selector::parse("ul.note-list li a").unwrap())
        .map(|link| link.text().collect())
        .collect();
    let bold = document
        .select(&Selector::parse("ul.note-list b").unwrap())
        .count();
    let raw_start = html.find("<li class=\"note-item\">").unwrap();
    let raw = &html[raw_start..raw_start + html[raw_start..].find("</a>").unwrap() +
4];

    println!("本文の中の形: {raw}");
    println!("リンクの文字: {links:?}");
    println!("一覧の中の b 要素: {bold} 個");
    assert_eq!(links, [title]);
    assert_eq!(bold, 0);
}
```

```
本文の中の形: <li class="note-item"><a href="/notes/1">&lt;b&gt;太字&lt;/b&gt; &amp; "引
用"</a>
リンクの文字: ["<b>太字</b> & \"引用\""]
一覧の中の b 要素: 0 個
```

- 本文の中では、`<` が `<`、`>` が `>`、`&` が `&` に書き換えられていました（Topcoat のエスケープ。第6巻）。`"` は、要素の中の文字なので、そのままです。

- `scraper` で読むと、リンクの文字は、入力した題と**まったく同じ**に戻り、`b` 要素は 0 個でした。「HTML としてではなく、ただの文字として表示された」ことを、文字の並びの細かい形に頼らずに確かめられます。

まとめ

- HTTP のテストでは、状態コード・ヘッダー（Location）・Set-Cookie（属性まで）・本文の4つを確かめる。まず全部を表示して、本当に返っているもの（と、返っていないもの）を見る。RustNote は、守りのヘッダーを返していない。
- 確かめの道具には `#[track_caller]` を付ける。付けると、失敗の場所が、道具の中ではなく、テストの呼んだ行になる。本文を読む `body_text` はレスポンスを受け取るので、状態コードは先に取っておく（E0382）。
- 同じ名前のヘッダーは `get_all` で全部読み、名前で探す。`Set-Cookie` の順番は決まっていない（cookie 0.18.2 の `HashSet`）ので、1つ目を読むテストは、ときどきだけ失敗する。本文は CSS セレクターで要素を決めて確かめる。`contains` は、検索の欄に戻した入力にも当たって、空の一覧でも通ってしまう。

UNIT 04: property test・fuzz・入力の読み取り

この単元のゴール

- 「どんな入力でも成り立つ決まり」（property）を関数で書き、proptest に、でたらめな入力を何百個も作らせて確かめられる
- 失敗した入力を、proptest が小さく縮める（shrink）様子と、失敗の種が `.proptest-regressions` に保存される仕組みを説明できる
- でたらめな入力を HTTP の入口に送り、RustNote の本物のバグ（NUL で始まる題で 500）を見つけ、原因（SQLite の `length()`）まで追える
- cargo-fuzz（WSL の nightly）で、コードの中を見ながら入力を変えていく fuzz を動かし、proptest が見逃した入力を見つける様子を確かめられる
- でたらめな試しは「見つけないこともある」ことを、実際の回数で説明できる

たとえ話でつかむ

新しい自動販売機の、出荷前の試験を考えます。

- **チェック表の試験（UNIT 01）**：人が「10 円玉だけ」「お札とおつりの境目」など、間違えやすい所を考えて、表のとおりに試します。考えた所は確実に試せますが、考えなかった所は試しません。
- **でたらめに押すロボット（property test）**：「どう押しても、入れたお金より多いおつりは出ない」という**決まり**を1つ決め、ロボットに、でたらめなボタンとお金で何百回も試させます。壊れたら、ロボットは「何をしたら壊れたか」を記録し、余分な操作を1つずつ取り除いて、**いちばん短い手順**にしてくれます（shrink）。
- **機械の中を見ながら押すロボット（fuzz）**：機械の中のどの部品が動いたかを見ながら、「今まで動かなかった部品が動いた押し方」を少しずつ変えて、何百万回も試します。
- どちらのロボットも、**運しだい**の所があります。今日の試験で見つからなかった不具合が、明日見つかることもあります。

自動販売機の試験	RustNote のテスト
チェック表の試験	UNIT 01 の表の形のテスト（人が選んだ端の値）
決まり	property（「合格・不合格が、決まりと同じ」など）
でたらめに押すロボット	proptest（入力の作り方 = strategy を決めて、でたらめに作る）
手順を短くする	shrink（失敗した入力を、小さく縮める）
機械の中を見ながら押すロボット	cargo-fuzz（libFuzzer。通ったコードの道筋を見て、入力を変える）
運しだい	見つかる回と、見つからない回がある（例4-3）

用語: property test（プロパティテスト：「どんな入力でも、この決まりが成り立つ」を、道具が作るたくさんの入力で確かめるテスト）

用語: strategy（ストラテジー：proptest で、入力の作り方を決めるもの。`"\\PC{0,100}"`（制御文字を除く 0〜100 文字）や `any::<u32>()` など）

用語: fuzz（ファズ：入力を少しずつ変えながら、とても多く試して、パニックやクラッシュを探すこと。それを行う道具を fuzzer と呼ぶ）

用語: NUL（ヌル：番号が 0 の文字。目には見えない。C 言語や SQLite では、文字列の終わりの印として使われることがある）

図で理解する

入力の選び方の、3つのやり方です。



図11-10: 入力の選び方。3つは置き換えではなく、組み合わせて使う。見つかった入力は、表の形のテストの1行として残す

例4-3で見つかったバグの仕組みです。

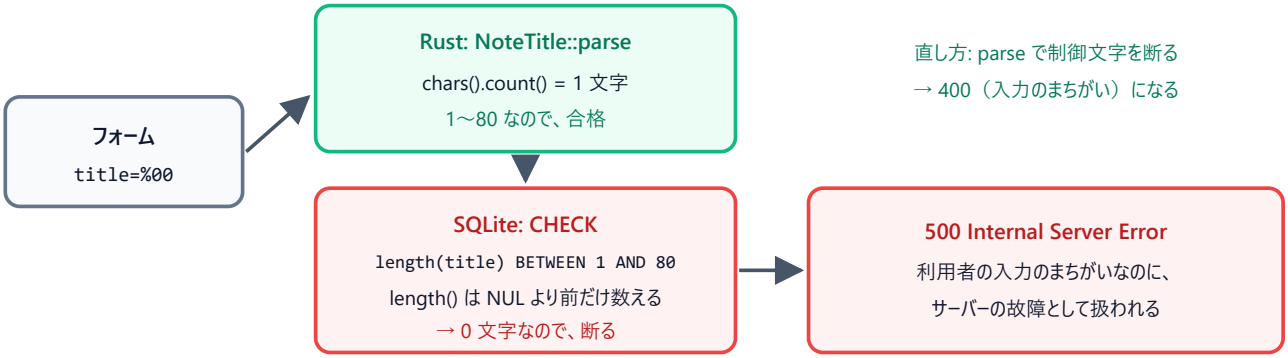


図11-11: NUL で始まる題の行方。Rust と SQLite で「文字数」の数え方が違うので、Rust の検査を通った題が、DB の CHECK で断られて 500 になる

まず動かす最小コード

例4-1: NoteTitle::parse の決まりを、でたらめな文字列で確かめる

proptest を、練習用のコピーの dev-dependency に足します。

```
cargo add --dev proptest@=1.12.0
```

```
    Updating crates.io index
      Adding proptest =1.12.0 to dev-dependencies
    Updating crates.io index
     Locking 15 packages to highest Rust 1.99 compatible versions
      Adding proptest v1.12.0
```

（`Features:` の一覧と、`proptest` が使う 14 個のクレートの `Adding` の行は、省いています。）

> **▼ このコードがやること（先に日本語で）**：決まり「前後の空白を除いた文字数が 1〜80 なら合格」を、`rule` という関数で書きます。1つ目のテストは、proptest がどんな文字列を作るのかを、5つだけ表示します（種を決めた `TestRunner::deterministic()` なので、毎回同じものが出ます）。2つ目は「`NoteTitle::parse` の合格・不合格が、`rule` と同じ」、3つ目は「合格した題を、もう一度 `parse` しても同じ題になる」を、それぞれ 256 個のでたらめな文字列で確かめます。

```rust
//! 第11巻 UNIT 04: NoteTitle::parse の決まりを、でたらめな文字列で、たくさん確かめる（property test）。
//!
//! 実行: cargo test --test u04_title_prop -- --nocapture

use proptest::prelude::*;
use proptest::strategy::ValueTree;
use proptest::test_runner::TestRunner;
use rustnote::domain::NoteTitle;

/// 決まり: 前後の空白を除いた文字数が 1〜80 なら合格
fn rule(input: &str) -> bool {
    (1..=80).contains(&input.trim().chars().count())
}

// 1. proptest が作る入力を、少しだけのぞいてみる（種を決めた TestRunner なので、毎回同じ）
#[test]
fn look_at_inputs() {
    let mut runner = TestRunner::deterministic();
    let strategy = "\\PC{0,100}";
    for _ in 0..5 {
        let input = strategy.new_tree(&mut runner).unwrap().current();
        println!(
            "{} 文字: {:?}…",
            input.chars().count(),
            input.chars().take(12).collect::<String>()
        );
    }
}

proptest! {
    // 2. どんな文字列でも、合格・不合格が、決まりと同じになる
    #[test]
    fn parse_follows_the_rule(input in "\\PC{0,100}") {
        prop_assert_eq!(NoteTitle::parse(&input).is_ok(), rule(&input));
    }

    // 3. 合格した題を、もう一度 parse しても、同じ題になる
    #[test]
    fn parse_twice_is_the_same(input in any::<String>()) {
        if let Ok(title) = NoteTitle::parse(&input) {
            prop_assert_eq!(NoteTitle::parse(title.as_str()), Ok(title));
        }
    }
}
```

```
cargo test --test u04_title_prop -- --nocapture --test-threads=1
```

▼ 実行結果

```
running 3 tests
test look_at_inputs ... 23 文字: "Ⲙ൩ᩚᩚ$vvFzl=ᴑ"…
5 文字: "6⬚ⷫ"…
69 文字: "ш\"\u{ccb}bjᵽz$?:ᷓo"…
41 文字: "`ᦟJ6´ᗝㄐl=▒İ¥ᐩ"…
98 文字: "/\\*=\u{748}=dಹᩚᩚY₳¥"…
ok
test parse_follows_the_rule ... proptest: FileFailurePersistence::SourceParallel set,
but failed to find lib.rs or main.rs
ok
test parse_twice_is_the_same ... proptest: FileFailurePersistence::SourceParallel set,
but failed to find lib.rs or main.rs
ok
test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.13s
```

（`test parse_… ...` の行のうしろに出る `proptest: FileFailurePersistence …` は、解説3で説明します。）

▼ コードを1つずつ分解して解説

解説1: 「答え」ではなく「決まり」を書く

- UNIT 01 の表は、「`"あ".repeat(80)` → 合格」のように、入力ごとに**答え**を書きました。property test では、入力が何かは分からないので、「合格・不合格は `rule` と同じ」のように、**どんな入力でも成り立つ決まり**を書きます。
- 決まりの書き方には、よく使う形があります。「別の方法で求めた答えと同じ」（`rule` と比べる）・「2回やっても同じ」（3つ目のテスト）・「元に戻せる」・「やってはいけないことが起きない」（パニックしない・500 にならない。例4-3）です。

解説2: 入力の作り方（strategy）

- `"\\PC{0,100}"` は、「制御文字などの目に見えない文字（Unicode の分類 `C`）以外の文字を、0〜100 個」という正規表現です。Rust の文字列の中で `\` を書くので、`\\` と2つ重ねます（エラー1）。
- 表示した5つのように、proptest は、いろいろな国の文字・絵文字・記号を混ぜた文字列を作ります。人がチェック表に書かない文字です。
- `proptest!` の中の `#[test]` は、何も指定しなければ、1つのテストにつき **256 個**の入力を試します。

解説3: 失敗の種を保存する場所

- 出力から省いた `FileFailurePersistence … failed to find lib.rs or main.rs` は、「失敗したときに、その入力の種（seed）を保存するファイルの場所を、`src/lib.rs` の隣で探したが、見つからなかった」という知らせです。`tests/` の中のテストでは、テストのファイルの隣（`tests/u04_title_prop.proptest-regressions`）に保存されます（例4-2）。失敗しなければ、ファイルは作られません。

もう一歩: 実務に近い形

例4-2: 失敗した入力を、小さく縮める（shrink）

UNIT 01 の「わざと間違えた版」を、property test で確かめます。

> ▼ **このコードがやること（先に日本語で）**：間違えた版を2つ用意します。`forgot_trim` は前後の空白を除き忘れた版、`counted_bytes` は文字数ではなくバイト数（`len()`）で数えた版です。それぞれが `rule` と同じになるかを、例4-1と同じ作り方の文字列で確かめます。

```
//! 第11巻 UNIT 04: わざと間違えた parse を、property test で確かめる。
//! 失敗した入力を、proptest が「もっと小さい入力」に縮めていく（shrink）様子を見る。
//!
//! 実行: cargo test --test u04_shrink -- --test-threads=1

use proptest::prelude::*;

/// 決まり: 前後の空白を除いた文字数が 1〜80 なら合格
fn rule(input: &str) -> bool {
    (1..=80).contains(&input.trim().chars().count())
}

/// わざと間違えた版1: 前後の空白を除き忘れた
fn forgot_trim(input: &str) -> bool {
    (1..=80).contains(&input.chars().count())
}

/// わざと間違えた版2: 文字数ではなく、バイト数で数えた
fn counted_bytes(input: &str) -> bool {
    (1..=80).contains(&input.trim().len())
}

proptest! {
    #[test]
    fn a_forgot_trim(input in "\\PC{0,100}") {
        prop_assert_eq!(forgot_trim(&input), rule(&input));
    }

    #[test]
    fn b_counted_bytes(input in "\\PC{0,100}") {
        prop_assert_eq!(counted_bytes(&input), rule(&input));
    }
}
```

```
cargo test --test u04_shrink -- --test-threads=1
```

▼ 実行結果（1回目）

```
test a_forgot_trim ... ok
test b_counted_bytes ... FAILED
proptest: Saving this and future failures in ...\rustnote\tests\u04_shrink.proptest-
regressions
cc 640bbddc2c5ae147af9f417d90f563ee4180e8dd3ecc56ad94c7d51b4b0d4999
thread 'b_counted_bytes' (213640) panicked at tests\u04_shrink.rs:23:1:
Test failed: assertion failed: `(left == right)`
  left: `false`,
 right: `true` at tests\u04_shrink.rs:31.
minimal failing input: input = "ʊ A0ㅓ0a  aA¡AA○ 巢AaCΣaɷ_a฿ㅿ 0®ᴅA†AA  ▫  ŋ\u{cc6}⇞"
    successes: 1
test result: FAILED. 1 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.07s
```

- `counted_bytes` の版が失敗し、proptest は `minimal failing input`（縮めた、失敗する入力）を表示しました。まだ長く見えますが、この間違いは「80 バイトを超え、80 文字以下」の文字列でしか表に出ないので、21 文字より短くは縮められません（1文字は多くても 4 バイトなので、20 文字では 80 バイトを超えません）。
  `successes: 1` は、1つ通ったあとの、2つ目の入力で、もう失敗したという意味です。
- `forgot_trim` の版は、**通ってしまいました**。これは、エラー3でくわしく見ます。
- `Saving this and future failures in …\tests\u04_shrink.proptest-regressions` のとおり、失敗の種が保存されました。中身です。

```
# Seeds for failure cases proptest has generated in the past. It is
# automatically read and these particular cases re-run before any
# novel cases are generated.
#
# It is recommended to check this file in to source control so that
# everyone who runs the test benefits from these saved cases.
cc 640bbddc2c5ae147af9f417d90f563ee4180e8dd3ecc56ad94c7d51b4b0d4999 # shrinks to input
= "ʊ A0ㅓ0a  aA¡AA○ 巢AaCΣaɷ_a฿ㅿ 0®ᴅA†AA  ▫  ŋ\u{cc6}⇞"
```

同じコマンドを、もう一度動かします。

▼ 実行結果（2回目）

```
test a_forgot_trim ... ok
test b_counted_bytes ... FAILED
minimal failing input: input = "ʊ A0ㅓ0a  aA¡AA○ 巢AaCΣaɷ_a฿ㅿ 0®ᴅA†AA  ▫  ŋ\u{cc6}⇞"
    successes: 0
test result: FAILED. 1 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.08s
```

- 2回目は、`successes: 0` のとおり、保存された種から、**同じ入力**が最初に試されて、同じように失敗しました。ファイルの先頭のコメントのとおり、このファイルを Git に入れておくと、ほかの人のパソコンや CI でも、同じ失敗を最初に試します。

例4-3: でたらめな入力を、HTTP の入口に送る

今度は、関数1つではなく、RustNote の**入口**（検索・ページ番号・作成のフォーム）に、でたらめな入力を送ります。決まりは「どんな入力でも、500 にならない」です。500 は「サーバーの故障」で、利用者の入力のまちがいなら、400 で返すのが約束です。

> **▼ このコードがやること（先に日本語で）**：alice を1人だけ登録し、300 回、次の2つを送ります。① `GET /notes?q=…&page=…`（検索の言葉は、でたらめなバイトを `%XX` の形にしたもの。ページ番号は、大きな数・負の数・数字でない文字）。② `POST /notes`（題と本文も、でたらめなバイト）。①は 200 か 400、②は 303 か 400 だけが返ることを確かめ、返った状態コードの回数を数えて表示します。proptest の `TestRunner` を自分で動かし、`tokio` の `Runtime` の `block_on` で、非同期の処理を待ちます。

```rust
//! 第11巻 UNIT 04: でたらめな入力を、HTTP の入口（検索・ページ番号・作成のフォーム）に送り、
//! どんな入力でも 500 にならず、決めた状態コードだけが返ることを確かめる。
//!
//! 実行: cargo test --test u04_http_prop -- --nocapture

mod common;

use std::cell::RefCell;
use std::collections::BTreeMap;

use common::test_app;
use proptest::prelude::*;
use proptest::test_runner::{Config, TestRunner};

/// バイトの並びを、全部 %XX の形にする（UTF-8 として正しくないバイトも送れる）
fn percent_all(bytes: &[u8]) -> String {
    bytes.iter().map(|byte| format!("%{byte:02X}")).collect()
}

/// ページ番号として送る文字列: 大きな数・負の数・数字でないもの
fn page_text() -> impl Strategy<Value = String> {
    prop_oneof![
        any::<u64>().prop_map(|n| n.to_string()),
        any::<i32>().prop_map(|n| n.to_string()),
        "[-+0-9a-z.]{0,8}",
    ]
}

#[test]
fn random_inputs_never_give_500() {
    let runtime = tokio::runtime::Runtime::new().unwrap();
    let app = runtime.block_on(test_app());
    let alice = runtime.block_on(app.sign_up("alice@example.com", "correct horse
battery"));
    let mut runner = TestRunner::new(Config::with_cases(300));
    // run に渡す関数は Fn（外の変数を書き換えられない）なので、RefCell に入れて数える
    let seen = RefCell::new(BTreeMap::new());

    let strategy = (
        proptest::collection::vec(any::<u8>(), 0..40),
        page_text(),
        proptest::collection::vec(any::<u8>(), 0..40),
        proptest::collection::vec(any::<u8>(), 0..40),
    );
    let result = runner.run(&strategy, |(q, page, title, body)| {
        runtime.block_on(async {
            // 一覧の検索とページ送り
            let uri = format!("/notes?q={}&page={page}", percent_all(&q));
            let status = app.get(&uri, Some(&alice)).await.status();
```

```
            *seen
                .borrow_mut()
                .entry(format!("GET /notes → {status}"))
                .or_insert(0) += 1;
            prop_assert!(
                status.as_u16() == 200 || status.as_u16() == 400,
                "{uri} → {status}"
            );

            // 作成のフォーム
            let form = format!("title={}&body={}", percent_all(&title),
percent_all(&body));
            let status = app.post_form("/notes", &form, Some(&alice)).await.status();
            *seen
                .borrow_mut()
                .entry(format!("POST /notes → {status}"))
                .or_insert(0) += 1;
            prop_assert!(
                status.as_u16() == 303 || status.as_u16() == 400,
                "{form} → {status}"
            );
            Ok(())
        })
    });
    for (what, count) in seen.borrow().iter() {
        println!("{what}: {count} 回");
    }
    result.unwrap();
}
```

```
cargo test --test u04_http_prop -- --nocapture
```

▼ 実行結果

```
GET /notes → 200 OK: 127 回
GET /notes → 400 Bad Request: 178 回
POST /notes → 303 See Other: 155 回
POST /notes → 400 Bad Request: 3 回
POST /notes → 500 Internal Server Error: 147 回
thread 'random_inputs_never_give_500' (218344) panicked at
tests\u04_http_prop.rs:75:12:
called `Result::unwrap()` on an `Err` value: Fail(Reason("title=%00&body= → 500
Internal Server Error at tests\\u04_http_prop.rs:65"), ([], "0", [0], []))
test random_inputs_never_give_500 ... FAILED
```

- **500 が出ました。** `title=%00&body= → 500 Internal Server Error` です。題が `%00`（NUL 1文字）のノートを作ると、RustNote が 500 を返しました。
- 最後の `([], "0", [0], [])` は、縮めた入力です（検索の言葉は空・ページは `"0"`・題は NUL の1バイト・本文は空）。でたらめな長い入力から始まって、「題が NUL 1バイト」という、いちばん小さい形まで縮めてくれました。
- `POST /notes → 500` が 147 回あるのは、縮める途中で、失敗する入力を何度も試したからです。このとき、標準エラー出力には、RustNote の `[error] サーバーの中でエラーが起きました <- DB の操作に失敗しました <- SQLite failure: CHECK constraint failed: length(title) BETWEEN 1 AND 80` という行が、147行出ていました（上の出力からは省いています）。

原因を、1つずつ確かめます。 題に NUL を含む4つの場合を、Rust・SQLite・HTTP の3か所で比べます（`tests/u04_nul.rs`）。

```rust
//! 第11巻 UNIT 04: property test が見つけた「題が NUL 文字だと 500」を、1つずつ確かめる。
//!
//! 実行: cargo test --test u04_nul -- --nocapture --test-threads=1

mod common;

use common::{encode, test_app};
use rustnote::domain::NoteTitle;

#[tokio::test]
async fn titles_with_nul() {
    let app = test_app().await;
    let alice = app
        .sign_up("alice@example.com", "correct horse battery")
        .await;
    let conn = app.db.db.connect().await.unwrap();

    for title in ["\0", "\0a", "a\0", "a\0b"] {
        // Rust の側: NoteTitle::parse の結果と、文字数
        let parsed = NoteTitle::parse(title).is_ok();
        let chars = title.chars().count();
        // SQLite の側: length() が数える文字数
        let mut rows = conn.query("SELECT length(?1)", [title]).await.unwrap();
        let sqlite: i64 = rows.next().await.unwrap().unwrap().get(0).unwrap();
        // HTTP: 作成のフォームで送る
        let form = format!("title={}&body=", encode(title));
        let status = app.post_form("/notes", &form, Some(&alice)).await.status();
        println!(
            "題 {title:?}: chars().count()={chars}・parse の合格={parsed}・SQLite の
length()={sqlite} → {status}"
        );
    }

    // おまけ: UTF-8 として読めないバイト（%FF と、途中で切れた「あ」の %E3%81）を送ると
    let status = app
        .post_form("/notes", "title=%FFa%E3%81&body=", Some(&alice))
        .await
        .status();
    let mut rows = conn
        .query(
            "SELECT title, hex(title) FROM notes ORDER BY id DESC LIMIT 1",
            (),
        )
        .await
        .unwrap();
    let row = rows.next().await.unwrap().unwrap();
    let stored: String = row.get(0).unwrap();
    let hex: String = row.get(1).unwrap();
```

```
    println!("題 %FFa%E3%81 → {status}・DB には {stored:?}（16 進数で {hex}）");
}
```

```
題 "\0": chars().count()=1・parse の合格=true・SQLite の length()=0 → 500 Internal Server
Error
題 "\0a": chars().count()=2・parse の合格=true・SQLite の length()=0 → 500 Internal
Server Error
題 "a\0": chars().count()=2・parse の合格=true・SQLite の length()=1 → 303 See Other
題 "a\0b": chars().count()=3・parse の合格=true・SQLite の length()=1 → 303 See Other
題 %FFa%E3%81 → 303 See Other・DB には "\u{fffd}a\u{fffd}"（16 進数で EFBFBD61EFBFBD）
```

- Rust の `chars().count()` は NUL も1文字と数えるので、`NoteTitle::parse` は4つとも合格させます。ところが SQLite の `length()` は、**NUL より前の文字だけ**を数えます（SQLite の決まり）。NUL で**始まる**題は `length()` が 0 になり、DB の `CHECK (length(title) BETWEEN 1 AND 80)` に断られます（図の2つ目）。RustNote は、DB の失敗をサーバーの故障として扱うので、500 になります。
- NUL が途中にある題（`"a\0"`・`"a\0b"`）は、`length()` が 1 なので、CHECK を通って保存されました。
- おまけの行のとおり、UTF-8 として読めないバイト（`%FF`・途中で切れた「あ」）は、断られずに、`\u{fffd}`（U+FFFD、16 進数で `EFBFBD`）に置き換えて受け取られました。

直した版で確かめます。練習用のコピーの `src/domain.rs` で、題に制御文字（NUL・タブなど）があれば断るようにします（教材の `Rust/rustnote` は変えていません）。

```
--- a/src/domain.rs
+++ b/src/domain.rs
@@ -214,6 +214,8 @@
     Empty,
     #[error("タイトルは{max}文字以内にしてください（現在{len}文字）")]
     TooLong { len: usize, max: usize },
+    #[error("タイトルに制御文字（NUL・タブなどの見えない文字）は使えません")]
+    Control,
 }

 /// 検査済みのタイトル（前後の空白を除いて 1〜80 文字）
@@ -237,6 +239,10 @@
                 max: Self::MAX_CHARS,
             });
         }
+        // SQLite の length() は NUL より前しか数えないので、NUL で始まる題は DB の CHECK で
断られる
+        if trimmed.chars().any(char::is_control) {
+            return Err(TitleError::Control);
+        }
         Ok(NoteTitle(trimmed.to_string()))
     }
```

```
題 "\0": chars().count()=1・parse の合格=false・SQLite の length()=0 → 400 Bad Request
題 "\0a": chars().count()=2・parse の合格=false・SQLite の length()=0 → 400 Bad Request
題 "a\0": chars().count()=2・parse の合格=false・SQLite の length()=1 → 400 Bad Request
題 "a\0b": chars().count()=3・parse の合格=false・SQLite の length()=1 → 400 Bad Request
題 %FFa%E3%81 → 303 See Other・DB には "�a�"（16 進数で EFBFBD61EFBFBD）
GET /notes → 200 OK: 73 回
GET /notes → 400 Bad Request: 227 回
POST /notes → 303 See Other: 45 回
POST /notes → 400 Bad Request: 255 回
test random_inputs_never_give_500 ... ok
```

- NUL を含む題は、4つとも 400（入力のまちがい）になり、でたらめな入力 300 回で、500 は 0 回でした。直した版でも、`cargo test --lib`（19 個）と `cargo test --test http`（14 個）は全部通りました。
- 直した版では、`POST /notes → 400` が 255 回に増えました。でたらめなバイトの多くに、制御文字（0〜31 番など）が入っているからです。

運しだい、ということ。直す前の版で、同じテスト（300 回）を 10 回くり返しました。

```
1 回目: 500 は見つからなかった
2 回目: title=%00&body= → 500 Internal Server Error
3 回目: 500 は見つからなかった
4 回目: 500 は見つからなかった
5 回目: 500 は見つからなかった
6 回目: title=%00&body= → 500 Internal Server Error
7 回目: title=%00&body= → 500 Internal Server Error
8 回目: 500 は見つからなかった
9 回目: 500 は見つからなかった
10 回目: 500 は見つからなかった
```

- 500 が見つかったのは、10 回のうち 3 回だけでした。題の最初のバイトが 0 になる入力は、1 回の試しで 256 分の 1 くらいしか作られないからです。見つからなかった7回は、`test random_inputs_never_give_500 ... ok` と、通ってしまいました。
- しかも、`TestRunner` を自分で動かしたこのテストでは、`proptest: FileFailurePersistence::SourceParallel set, but no source file known` と表示され、**失敗の種は保存されませんでした**（`proptest!` で書いたテストとは違います）。だから、見つかった入力は、`tests/u04_nul.rs` のような**ふつうのテスト**として残します。次からは、毎回、必ず確かめられます。

例4-4: fuzz（cargo-fuzz）で、もっとたくさん試す

fuzz には、cargo-fuzz を使います。cargo-fuzz は、libFuzzer という fuzzer を使い、Rust の **nightly**（毎晩作られる、試しの版）が要ります。Windows（ARM64）ではなく、WSL の Ubuntu（aarch64）で動かしました。

```
# WSL（Ubuntu）の中で（1回だけ）
rustup toolchain install nightly --profile minimal
cargo install cargo-fuzz --locked
```

```
$ cargo +nightly fuzz --version
cargo-fuzz 0.13.2
$ rustc +nightly --version
rustc 1.101.0-nightly (92b5b8c87 2026-10-29)
aarch64
```

RustNote 全体は、DB（libSQL の C のコード）まで組み立てる必要があるので、ここでは、検査済みの型の `src/domain.rs` だけを取り込む、小さな練習用のクレート（ `fuzz_lab` ）を作ります。`domain.rs` は、標準ライブラリと `thiserror` だけを使うので、そのまま取り込めます。

```
[package]
name = "domain-lab"
version = "0.1.0"
edition = "2024"
publish = false

# RustNote の src/domain.rs だけを取り込んで fuzz するための、練習用のクレート
[dependencies]
thiserror = "=2.0.21"
```

```
//! RustNote の検査済みの型（src/domain.rs）を、そのまま取り込む。
//! domain.rs は標準ライブラリと thiserror だけを使うので、DB や Topcoat なしで動かせる。

#[path = "../../rustnote/src/domain.rs"]
pub mod domain;
```

```
cd fuzz_lab
cargo +nightly fuzz init --target parsers
cargo +nightly fuzz add query_offset
```

`fuzz/fuzz_targets/` に、fuzz の対象（fuzz target）を2つ書きます。

> **▼ このコードがやること（先に日本語で）：** `parsers` は、fuzzer が作る文字列を、RustNote の検査済みの型の `parse` に次々に渡し、パニックしないことと、「合格した値を、もう一度 `parse` しても同じ」を確かめます。 `query_offset` は、`NoteQuery` を**項目を直接書いて**作り（ `page` は `pub` ）、`offset` がパニックしないかを試します。

```
//! 第11巻 UNIT 04: RustNote の検査済みの型の parse に、fuzzer が作るでたらめな文字列を、次々に渡す。
//! パニックしないことと、「合格した値を、もう一度 parse しても同じ」を確かめる。
#![no_main]

use domain_lab::domain::{Email, NoteBody, NoteId, NoteQuery, NoteTitle, Password};
use libfuzzer_sys::fuzz_target;

fuzz_target!(|input: &str| {
    if let Ok(title) = NoteTitle::parse(input) {
        assert_eq!(NoteTitle::parse(title.as_str()), Ok(title));
    }
    if let Ok(email) = Email::parse(input) {
        assert_eq!(Email::parse(email.as_str()), Ok(email));
    }
    let _ = NoteBody::parse(input);
    let _ = Password::parse(input);
    let _ = input.parse::<NoteId>();
    let query = NoteQuery::new(Some(input), None);
    let _ = query.offset();
});
```

```
//! 第11巻 UNIT 04: NoteQuery を、項目を直接書いて作り（page は pub）、offset がパニックしないかを試す。
//! proptest の any::<u32>() 256 回では、見つからなかった入力を、fuzzer は見つけられるか。
#![no_main]

use domain_lab::domain::NoteQuery;
use libfuzzer_sys::fuzz_target;

fuzz_target!(|page: u32| {
    let query = NoteQuery {
        keyword: None,
        page,
    };
    let _ = query.offset();
});
```

`parsers` を 60 秒だけ動かします。

```
cargo +nightly fuzz run parsers -- -max_total_time=60
```

```
INFO: Seed: 1219589596
INFO: A corpus is not provided, starting from an empty corpus
#2  INITED cov: 84 ft: 85 corp: 1/1b exec/s: 0 rss: 32Mb
#32768  pulse  cov: 269 ft: 687 corp: 225/2567b lim: 53 exec/s: 16384 rss: 59Mb
#65536  pulse  cov: 275 ft: 803 corp: 285/5825b lim: 135 exec/s: 13107 rss: 85Mb
#131072 pulse  cov: 276 ft: 859 corp: 319/8994b lim: 341 exec/s: 16384 rss: 134Mb
#262144 pulse  cov: 276 ft: 901 corp: 345/13534b lim: 958 exec/s: 18724 rss: 241Mb
#524288 pulse  cov: 276 ft: 918 corp: 356/15894b lim: 2688 exec/s: 21845 rss: 462Mb
#1048576    pulse  cov: 283 ft: 963 corp: 385/21Kb lim: 4096 exec/s: 27594 rss: 519Mb
#1847970    DONE   cov: 289 ft: 1009 corp: 418/21Kb lim: 4096 exec/s: 30294 rss: 519Mb
Done 1847970 runs in 61 second(s)
```

- 61 秒で、1,847,970 回試して、パニックは 0 回でした。`cov`（通ったコードの場所の数）は 289 まで増え、`corp`（「新しい場所を通った入力」として取っておいた入力）は 418 個になりました。fuzzer は、この取っておいた入力を少しずつ変えて、次の入力を作ります。
- 例4-3の NUL のバグは、この fuzz では見つかりません。`domain.rs` だけを取り込んだので、DB の CHECK は通らないからです。fuzz は、**取り込んだ範囲の中**だけを調べます。

`query_offset` を動かします（見つかったら、そこで止まります）。

```
cargo +nightly fuzz run query_offset
```

```
#2  INITED exec/s: 0 rss: 31Mb
thread '<unnamed>' (198) panicked at fuzz_lab/src/../../rustnote/src/domain.rs:370:19:
attempt to subtract with overflow
SUMMARY: libFuzzer: deadly signal
MS: 2 ChangeByte-CMP- DE: "\000\000\000\000"-; base unit:
30a43079862afcbbee48b9c3416157b655454a44
Failing input:
    fuzz/artifacts/query_offset/crash-87fc293da8f8dc8a90eb1c7141afe4533fc003dc
Output of `std::fmt::Debug`:
    0
Reproduce with:
    cargo fuzz run query_offset fuzz/artifacts/query_offset/crash-
87fc293da8f8dc8a90eb1c7141afe4533fc003dc
```

- `domain.rs:370:19` の `attempt to subtract with overflow`（引き算で、桁があふれた）で止まりました。370 行目は `u64::from(self.page - 1) * u64::from(PER_PAGE)` です。`Output of std::fmt::Debug: 0` のとおり、`page` が **0** のとき、`0 - 1` が `u32` で表せず、パニックします（fuzz の組み立ては、debug と同じく、あふれを確かめます）。
- `MS: 2 ChangeByte-CMP- DE: "\000\000\000\000"` は、入力の変え方の記録です。`CMP` は、「コードの中で比べている値（ここでは 0）を、入力に入れてみた」という意味です。fuzzer は、コードの中を見ているので、`0` のような値を、すぐに試せます。

- 同じことを、proptest の `any::<u32>()`（256 回）でも確かめていました（`tests/u04_query_prop.rs`）。

```
test offset_from_fields ... ok
test offset_from_new ... ok
test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.01s
```

- proptest は、`page = 0` を見つけられず、**通って**しまいました。`u32` は 42 億通りあるので、256 回のでたらめでは、0 にまず当たりません。
- 今の RustNote は、`NoteQuery` を、`page` を 1 以上にそろえる `NoteQuery::new` でしか作っていないので、このパニックは起きません（問3）。ただし、項目が `pub` なので、だれかが `NoteQuery { page: 0, .. }` と書けば起きます。**型が、約束（page は 1 以上）を守っていない**ことを、fuzz が見つけました（改善点）。

よくあるエラーと読み方

エラー1: 正規表現の `\` を、1つだけ書いた

```
//! 第11巻 UNIT 04 エラー1: 正規表現の \ を、Rust の文字列の中で1つだけ書いた。
//! 実行: cargo test --test e04_escape

use proptest::prelude::*;
use rustnote::domain::NoteTitle;

proptest! {
    #[test]
    fn parse_never_panics(input in "\PC{0,100}") {
        let _ = NoteTitle::parse(&input);
    }
}
```

```
   Compiling rustnote v0.1.0 (...\rustnote)
error: unknown character escape: `P`
 --> tests\e04_escape.rs:9:38
  |
9 |     fn parse_never_panics(input in "\PC{0,100}") {
  |                                      ^ unknown character escape
  |
  = help: for more information, visit <https://doc.rust-
lang.org/reference/tokens.html#literals>
help: if you meant to write a literal backslash (perhaps escaping in a regular
expression), consider a raw string literal
  |
9 |     fn parse_never_panics(input in r"\PC{0,100}") {
  |                                     +
error[E0277]: the size for values of type `str` cannot be known at compilation time
error[E0277]: the size for values of type `str` cannot be known at compilation time
error: could not compile `rustnote` (test "e04_escape") due to 3 previous errors
```

- **何が起きたか:** Rust の `"…"` の中では、`\` は「次の文字と組で、特別な文字を表す」印です（`\n` が改行、`\t` がタブ）。`\P` という組は無いので、`unknown character escape`（知らない組み合わせ）になりました。続く2つの E0277 は、最初のエラーのせいで出た、おまけのエラーです。最初のエラーから直します。
- **直し方:** `"\\PC{0,100}"` のように `\` を2つ重ねるか、`help:` のとおり `r"\PC{0,100}"`（`r` を付けた、`\` をそのまま書ける文字列）にします。

エラー2: `run` に渡す関数の中で、外の変数を書き換えた（E0596）

例4-3の数えるための `seen` を、`RefCell` に入れずに `let mut seen = BTreeMap::new();` と書いた場合です。

```
   Compiling rustnote v0.1.0 (...\rustnote)
error[E0596]: cannot borrow `seen` as mutable, as it is a captured variable in a `Fn`
closure
  --> tests\e04_fnmut.rs:43:26
   |
34 |       let mut seen = BTreeMap::new();
   |           -------- `seen` declared here, outside the closure
...
42 |       let result = runner.run(&strategy, |(q, page, title, body)| {
   |                           -              ----------------------- in this closure
   |  _________________________|
   | |
43 | |         runtime.block_on(async {
   | |                          ^^^^^ cannot borrow as mutable
44 | |             // 一覧の検索とページ送り
45 | |             let uri = format!("/notes?q={}&page={page}", percent_all(&q));
46 | |             let status = app.get(&uri, Some(&alice)).await.status();
47 | |             *seen.entry(format!("GET /notes → {status}")).or_insert(0) += 1;
   | |              ---- mutable borrow occurs due to use of `seen` in closure
...  |
62 | |         })
63 | |     });
   | |______- expects `Fn` instead of `FnMut`

For more information about this error, try `rustc --explain E0596`.
error: could not compile `rustnote` (test "e04_fnmut") due to 1 previous error
```

- **何が起きたか**: `TestRunner::run` は、渡す関数を `Fn`（外の変数を、読むだけの関数）として受け取ります（`expects Fn instead of FnMut`）。その中で `seen` を書き換えようとしたので、「`Fn` のクロージャーの中で、`seen` を書き換えのために借りられない」と言われました（第3巻の借用の決まり）。
- **直し方**: 例4-3のように `RefCell::new(BTreeMap::new())` に入れ、使うときに `seen.borrow_mut()` で書き換えます。`RefCell` は、「書き換えのための借用を、コンパイルのときではなく、動かしているときに確かめる」入れ物です（第3巻 UNIT 09）。

エラー3（通ってしまう）：前後の空白を除き忘れた版を、見つけられなかった

例4-2の `forgot_trim` は、間違えた版なのに、256 回の試しを**通りました**。

- **なぜか**: `forgot_trim` が `rule` と違う答えを出すのは、「空白だけの文字列」か、「前後に空白があって、ちょうど 80 文字の境目をまたぐ文字列」のときだけです。`"\\PC{0,100}"` は、いろいろな国の文字を、まんべんなく作るので、空白だけの文字列は、ほとんど作られません。
- **直し方**: 間違いが出やすい形に、作り方を**寄せます**。空白（半角・全角・タブ）と `a` だけで、0〜85 文字の文字列を作ります（`tests/u04_spaces.rs`）。

```rust
//! 第11巻 UNIT 04: 入力の作り方を、間違いが出やすい形に寄せる。
//! 空白（半角・全角・タブ）と「a」だけで、0〜85 文字の文字列を作る。
//!
//! 実行: cargo test --test u04_spaces

use proptest::prelude::*;

/// 決まり: 前後の空白を除いた文字数が 1〜80 なら合格
fn rule(input: &str) -> bool {
    (1..=80).contains(&input.trim().chars().count())
}

/// わざと間違えた版: 前後の空白を除き忘れた
fn forgot_trim(input: &str) -> bool {
    (1..=80).contains(&input.chars().count())
}

proptest! {
    #[test]
    fn forgot_trim_with_spaces(input in "[ \t\u{3000}a]{0,85}") {
        prop_assert_eq!(forgot_trim(&input), rule(&input));
    }
}
```

3回動かしました（毎回、保存された種を消してから）。

```
minimal failing input: input = "a  \t\u{3000}\t\u{3000}aa\t\t\u{3000}\t\u{3000}\u{3000}
\u{3000}\t\t\u{3000}a\t
a\u{3000}\u{3000}\t\t\u{3000}a\ta\u{3000}\u{3000}\u{3000}\t\u{3000}\t\u{3000}\t a
\taaa\u{3000}\t a a\t\u{3000}\t\u{3000} \u{3000} a\u{3000}\u{3000}\ta\u{3000}  a  \ta
\u{3000}\t a\u{3000}\t"
    successes: 9
```

- 3回とも見つかりました。`successes` は 9・0・37 で、10 個目・1 個目・38 個目の入力で失敗しました（上は1回目）。3回とも、81 文字（前後の空白を除くと 78〜80 文字）の文字列まで縮みました。`forgot_trim` は 81 文字なので不合格、`rule` は前後の空白を除くと 80 文字以下なので合格、という「80 文字の境目」での食い違いです。
- この作り方で試した別の回には、空白1つの `" "` や、タブ1つの `"\t"`（空白だけの食い違い）まで縮んだこともありました。どちらの食い違いを先に見つけたかで、縮み方が変わります。
- **学べること:** property test は、作り方の**外**の入力は試しません。「どんな入力で間違いが出るか」を考えて、作り方を決めます。そして、UNIT 01 の表（空白だけ・80 と 81）も、残しておきます。表は、毎回、必ず同じ所を試すからです。

RustNote ではここで使う

- **見つかったバグ**: 題が NUL で始まるノートを作ると 500 になります（例4-3）。利用者は、ふつうのブラウザの入力欄からは NUL を入れにくいですが、フォームを自分で作れば、だれでも送れます。教材の `Rust/rustnote` のコードは変えていません。直すなら、例4-3の差分のように、`NoteTitle::parse` で制御文字を断ります（本文の `NoteBody` には CHECK が無いので、NUL が入っても 500 にはなりません）。
- **型が守っていない約束**: `NoteQuery` の項目は `pub` なので、`page = 0` で作るとパニックします（例4-4）。項目を `pub` でなくし、`NoteQuery::new` だけで作れるようにすると、型が約束を守れます（改善点）。
- **読めないバイト**: UTF-8 として読めないバイトは、断られずに `` に置き換えて受け取られます。断りたいときは、`parse` で `\u{FFFD}` を断る方法があります（今は受け付けています）。
- RustNote の `tests/` には、property test も fuzz もありません。この単元のテストは、練習用のコピーにだけ足しました（`Cargo.toml` の `proptest` も同じ）。

練習問題

問1: `Email::parse` について、「合格したアドレスを、もう一度 `parse` しても同じ」と「合格したアドレスには、空白も大文字も無く、`@` はちょうど1つ」を、property test で確かめてください。でたらめな文字列では、ほとんど合格しないので、「メールアドレスらしい形」の作り方を考えてください。

問2: 例4-3の、直した版の結果では、`GET /notes → 400` が 227 回、`200` が 73 回でした。検索の言葉は、どんなバイトでも 200 になります。では、400 は、どの入力から来たのでしょうか。`page_text()` の作り方から考えてください。

問3: 例4-4の `query_offset` を、URL から作るときと同じ `NoteQuery::new(None, page)`（`page` は `Option<u32>`）で作る版にして、30 秒動かすと、どうなるでしょうか。

解答と解説

問1の解答:

```rust
//! 第11巻 UNIT 04 問1: Email::parse の決まりを、property test で確かめる。
//! 実行: cargo test --test u04_q1 -- --nocapture

use proptest::prelude::*;
use rustnote::domain::Email;

/// メールアドレスらしい形（大文字・空白・全角も混ぜる）
fn email_like() -> impl Strategy<Value = String> {
    "[ ]{0,2}[a-zA-Z0-9.Ａ_]{0,10}@[a-zA-Z0-9.Ｂ-]{0,10}[ ]{0,2}"
}

proptest! {
    // 合格したアドレスを、もう一度 parse しても同じ
    #[test]
    fn parse_twice_is_the_same(input in email_like()) {
        if let Ok(email) = Email::parse(&input) {
            prop_assert_eq!(Email::parse(email.as_str()), Ok(email));
        }
    }

    // 合格したアドレスには、空白も大文字も無く、@ はちょうど1つ
    #[test]
    fn accepted_addresses_are_tidy(input in email_like()) {
        if let Ok(email) = Email::parse(&input) {
            let text = email.as_str();
            prop_assert!(!text.contains(char::is_whitespace));
            prop_assert_eq!(text, text.to_lowercase());
            prop_assert_eq!(text.matches('@').count(), 1);
        }
    }
}

// どのくらいの割合が合格するかも見ておく（合格が少なすぎると、上の確かめが空回りする）
#[test]
fn how_many_are_accepted() {
    use proptest::strategy::ValueTree;
    use proptest::test_runner::TestRunner;
    let mut runner = TestRunner::deterministic();
    let strategy = email_like();
    let mut accepted = 0;
    for _ in 0..1000 {
        let input = strategy.new_tree(&mut runner).unwrap().current();
        if Email::parse(&input).is_ok() {
            accepted += 1;
        }
    }
    println!("1000 個のうち、合格は {accepted} 個");
}
```

```
test accepted_addresses_are_tidy ... proptest: FileFailurePersistence::SourceParallel
set, but failed to find lib.rs or main.rs
ok
test how_many_are_accepted ... 1000 個のうち、合格は 179 個
ok
test parse_twice_is_the_same ... proptest: FileFailurePersistence::SourceParallel set,
but failed to find lib.rs or main.rs
ok
test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.15s
```

- 2つの property は、どちらも 256 回で通りました。`how_many_are_accepted` のとおり、この作り方では 1000 個のうち 179 個が合格します。合格する入力が少なすぎると、`if let Ok(…)` の中の確かめが、ほとんど動かずに「通って」しまうので、割合も見ておきます。
- 作り方には、前後の空白・大文字・全角の文字（`A`・`B`）・`.` や `-` を混ぜました。`parse` が、空白を除いて小文字にそろえることを、確かめるためです。

問2の解答:

- `page_text()` は、3つの作り方から1つを、同じくらいの割合で選びます。① `any::<u64>()` の数: ほとんどが `u32` の上限（約 42 億）より大きいので、`?page=` を `u32` として読めず 400。② `any::<i32>()` の数: 半分くらいが負の数なので 400。③ `"[-+0-9a-z.]{0,8}"`: 数字だけになることは少なく、多くは 400。
- 全部を合わせると、だいたい 4 分の 3 くらいが 400 になり、300 回中 227 回と合います。ページ番号のまちがいは、500 ではなく 400 になる、ということも、同時に確かめられました。

問3の解答:

```
//! 第11巻 UNIT 04 問3: NoteQuery を、URL から作るときと同じ NoteQuery::new で作り、offset を
試す。
#![no_main]

use domain_lab::domain::NoteQuery;
use libfuzzer_sys::fuzz_target;

fuzz_target!(|page: Option<u32>| {
    let query = NoteQuery::new(None, page);
    let _ = query.offset();
});
```

```
cargo +nightly fuzz run query_new -- -max_total_time=30
```

```
#10639929   DONE   cov: 45 ft: 47 corp: 4/8b lim: 4096 exec/s: 343223 rss: 605Mb
Done 10639929 runs in 31 second(s)
```

- 31 秒で 10,639,929 回試して、パニックは 0 回でした。`NoteQuery::new` は、`page.unwrap_or(1).max(1)` で、0 を 1 にそろえるからです。`cov` が 45 で止まったのは、調べる場所が少ないからです。
- 同じ `offset` でも、作り方の入口（`new` か、項目を直接書くか）で、結果が変わります。入口を `new` の1つにしぼれば、約束を型が守れます。

まとめ

- property test は、「どんな入力でも成り立つ決まり」を書き、proptest に、でたらめな入力を何百個も作らせて確かめる。失敗すると、入力を小さく縮め、`proptest!` なら種を `.proptest-regressions` に保存する。決まりは「別の方法と同じ」「2回やっても同じ」「500 にならない・パニックしない」などの形で書く。
- でたらめな入力を HTTP の入口に送ると、題が NUL で始まると 500 になる、本物のバグが見つかった。Rust は NUL も1文字と数え、SQLite の `length()` は NUL より前だけを数えるので、DB の CHECK で断られる。ただし、見つかったのは 10 回中 3 回。見つかった入力は、ふつうのテストとして残す。
- cargo-fuzz（WSL の nightly）は、コードの中を見ながら、1 分で 100 万回以上試す。`NoteQuery { page: 0 }` のパニックを、proptest の 256 回が見逃したのに、すぐに見つけた。property test は作り方の外を試さないので、間違いが出やすい形に作り方を寄せ、UNIT 01 の表も残す。

この巻のまとめ

この版に収録した UNIT 01〜04 で、次のことができるようになったかを確かめましょう。できたものに印を付けてください。

UNIT 01: ユニットテスト・境界値・表の形のテスト

- ☐ テストの段（ユニット・DB の結合・HTTP・E2E）の違いと、細かい決まりを下の段で確かめる理由を説明できる
- ☐ 「1〜80 文字」のような決まりから、境界値（0・1・80・81）と、間違えやすい入力（空白・全角の空白・日本語・絵文字）を選べる
- ☐ 名前付きの行を表に並べ、1つの確かめ方を全部の行に当てて、合わない行をまとめて報告するテストを書ける
- ☐ わざと間違えた版に同じ表を当てて、テストが失敗することを確かめられる。短い日本語だけの表では `len()` の間違いを見つけられない理由を言える
- ☐ 文字（`chars().count()`）・バイト（`len()`）・UTF-16 の単位の3つの数え方の違いを、絵文字と「が」で説明できる
- ☐ `assert_eq!` の失敗の表示（`left` と `right`）を読み、表と関数のどちらが間違っているかを考えられる
- ☐ `#[test]` を async fn に付けたエラーと、結果を確かめないテストが何も守っていないことを説明できる

UNIT 02: リポジトリの結合テスト

- ☐ リポジトリを、偽物ではなく本物の DB（一時フォルダの libSQL ファイル）で確かめる理由を、`LIKE` の `ESCAPE` の例で説明できる
- ☐ 準備・操作・確かめの形で、成功・見つからない・他人・版の食い違いを、1つずつ確かめるテストを書ける
- ☐ SQL から `ESCAPE` を消すと、ユニットテストは通ったまま、結合テストだけが失敗することを確かめられる
- ☐ 削除の SQL から持ち主の条件を消したときの失敗を読み、`assert!` に実際の値を出す文を足せる
- ☐ 1つの DB ファイルを使い回したテストが、実行のたびに違う理由（UNIQUE・locked・表がもうある）で失敗することと、`common::test_db()` で直す理由を言える
- ☐ 検索の大文字と小文字・セッションの期限ちょうど・ユーザーを消したときの CASCADE を、本物の DB で確かめられる

UNIT 03: HTTP のテスト

- ☐ レスポンスの4つの部分（状態コード・ヘッダー・Set-Cookie・本文）を全部表示して読み、それぞれ何を確かめればよいかを言える
- ☐ assert_redirect や cookie_attributes のような確かめの道具を作り、#[track_caller] で、失敗の場所をテストの行にできる
- ☐ 本文を contains ではなく、scraper の CSS セレクターで要素を決めて確かめる理由を、空の一覧でも通ってしまう例で説明できる
- ☐ 同じ名前のヘッダーを get_all で全部読み、名前で探す理由（Set-Cookie の順番は決まっていない）を言える
- ☐ body_text のあとでレスポンスを使えない E0382 を読み、状態コードを先に取っておく形に直せる

UNIT 04: property test・fuzz・入力の読み取り

- ☐「どんな入力でも成り立つ決まり」を関数で書き、proptest にでたらめな入力を作らせて確かめられる
- ☐ 失敗した入力が縮められる（shrink）様子と、種が .proptest-regressions に保存される仕組みを説明できる
- ☐ でたらめな入力を HTTP の入口に送り、500 にならないことを確かめ、見つかった入力（NUL で始まる題）の原因を追える
- ☐ cargo-fuzz（WSL の nightly）で fuzz target を書いて動かし、見つかった入力を cargo fuzz fmt で読める
- ☐ でたらめな試しは見つけない回もあることと、作り方を間違いの出やすい形に寄せる理由を言える

> **お知らせ:** この版には UNIT 01〜04 を収録しています。UNIT 05（E2E・ブラウザ・アクセシビリティ）・UNIT 06（守りの回帰テスト）・UNIT 07（tracing・メトリクス・health・SLO）・UNIT 08（benchmark・profile・メモリの使い方）・UNIT 09（負荷試験・背圧・容量）・UNIT 10（CI・依存・供給網・事故の対応）は準備中で、次の更新から順に追加します。それまでの間、RustNote のテストは、`Rust/rustnote/tests/http.rs` と `tests/repository.rs`、`src/` の各ファイルの最後の `#[cfg(test)]` で読めます。

用語ミニ辞典

この巻（UNIT 01〜04）に出てきた用語です。

UNIT 01 の用語

用語	意味
ユニットテスト	関数や型のような小さな部品1つを、ほかから切り離して確かめるテスト
テストの段	ユニット・DB の結合・HTTP・E2E のように、確かめる範囲の広さで分けたテストの種類
境界値	決まりが切り替わる境目の値。端と、その1つ外側を試す
表の形のテスト（table-driven test）	入力と期待する結果の組を表に並べ、同じ確かめ方で1行ずつ試すテスト
`#[cfg(test)]`	`cargo test` のときだけコンパイルする印。本番の実行ファイルには入らない
`assert_eq!`	2つの値が等しいことを確かめる。違えば `left` と `right` を表示して panic する
わざと間違えた版	テストが間違いを見つけられるかを確かめるために、わざと壊した関数
ミューテーションテスト	わざと間違いを入れて、テストが気づくかを道具で自動に確かめる方法
ゼロ幅接合子（U+200D）	絵文字どうしをつないで1つに見せる、見えない文字
UTF-16 の単位	文字を 16 ビットずつで表したときの数。ブラウザの `maxlength` はこれで数える
`#[tokio::test]`	async のテストに付ける印。Tokio のランタイムを用意してから動かす

UNIT 02 の用語

用語	意味
結合テスト	いくつかの部品を、本物どうしでつないで確かめるテスト。ここでは、リポジトリと本物の DB
Arrange・Act・Assert	テストを、準備する・操作する・確かめるの3つに分けて書く形
偽物（fake）	テストのために作った、本物の代わりの実装。SQL は動かないので、SQL の間違いには気づけない
`common::test_db()`	テストごとに一時フォルダを作り、マイグレーション済みの空の DB を渡す RustNote の関数
`TempDir`	`tempfile` クレートの一時フォルダ。片付けられる（drop される）と、フォルダごと消える
`ESCAPE '\'`	`LIKE` の中で、`\` のうしろの文字（`%`・`_`）を、ただの文字として扱わせる指定
`VersionMismatch`	更新しようとした version が、今の version と違ったときのエラー（楽観的ロック）
共有しているもの	DB のファイル・ポート番号・環境変数・今の時刻など、テストどうしで同じものを使う所。ときどき失敗するテストの原因になる
`database is locked`	ほかの接続が書いている最中で、DB を使えなかったときの SQLite のエラー
`ON DELETE CASCADE`	参照先の行が消えたら、参照している行も一緒に消す、という表の約束

UNIT 03 の用語

用語	意味
ヘッダー	レスポンスの本文の前に付く「名前: 値」の行。同じ名前の行が、いくつあってもよい
`Set-Cookie`	クッキーを渡すヘッダー。`HttpOnly`・`Secure`・`SameSite`・`Path`・`Max-Age` などの属性が付く
CSS セレクター	`ul.note-list li a` のように、HTML の要素を場所で指定する書き方
`scraper`	HTML を読み、CSS セレクターで要素を探すクレート
`#[track_caller]`	関数の中で失敗したとき、その関数を呼んだ行を、失敗の場所として表示させる印
`get_all`	同じ名前のヘッダーを、全部取り出すメソッド（`get` は1つ目だけ）
守りのヘッダー	`X-Frame-Options`・`Content-Security-Policy` など、ブラウザに守り方を伝えるヘッダー

UNIT 04 の用語

用語	意味
property test	どんな入力でも成り立つ決まりを、道具が作るたくさんの入力で確かめるテスト
proptest	Rust の property test のクレート。`proptest!` と strategy で入力を作る
strategy	入力の作り方。正規表現の文字列・`any::<u32>()`・`prop_oneof!` など
shrink	失敗した入力を、失敗したまま、小さく縮めること
`.proptest-regressions`	失敗した入力の種を保存するファイル。Git に入れて共有する
fuzz・fuzzer	入力を少しずつ変えて、とても多く試し、パニックを探すこと・その道具
cargo-fuzz	libFuzzer を使う Rust の fuzz の道具。nightly で動く
NUL	番号が 0 の文字。SQLite の `length()` は NUL より前だけを数える